INTERVIEW PREPARATION HANDBOOK

100 Intermediate-to-Difficult AI Agent Interview Questions

Detailed model answers for architecture, planning, tools, memory, security, evaluation, scaling, and production bottlenecks

100	10	Production
scenario-based questions	technical domains	trade-offs and failure modes

Designed for senior AI engineer, applied AI, agent platform, and LLM systems interviews

Contents

Each section contains ten scenario-based questions with a model answer, implementation choices, common bottlenecks, validation criteria, and the signal interviewers typically seek.

Section	Domain	Questions
1	**Agent Architecture and Design Decisions**	1-10
2	**Planning, Reasoning, and Task Decomposition**	11-20
3	**Tool Selection and Tool Execution**	21-30
4	**Agent State, Memory, and Context Management**	31-40
5	**Agent Loops, Termination, and Recovery**	41-50
6	**Multi-Agent Systems**	51-60
7	**Human-in-the-Loop and Authorization**	61-70
8	**Security, Safety, and Trust Boundaries**	71-80
9	**Evaluation, Testing, and Observability**	81-90
10	**Performance, Scaling, Cost, and Bottlenecks**	91-100

Recommended interview structure: Clarify requirements first, state the control model, draw the architecture, walk through the happy path, then cover failure modes, security boundaries, observability, and measurable success criteria.

How to Use This Handbook

Do not memorize the answers word for word. Use the recurring decision framework to produce a structured response that can be adapted to the company, risk level, latency target, and degree of autonomy in the scenario.

A strong system-design answer follows six moves

1. Clarify the goal, user, business impact, autonomy boundary, latency target, cost ceiling, and compliance constraints.
2. Choose the control structure: deterministic workflow, bounded agent, planner-executor loop, graph, or multi-agent system.
3. Separate trusted control logic from probabilistic model decisions and untrusted external content.
4. Define state, tool contracts, authorization, approvals, idempotency, retries, timeouts, and termination conditions.
5. Design observability and evaluation around task success, path quality, safety, latency, cost, and recoverability.
6. Explain graceful degradation, rollout, incident response, and how the design evolves as traffic and risk grow.

Reference production architecture

A robust agent platform usually has a deterministic control plane around probabilistic model calls. The exact components vary, but the trust boundaries remain similar.

1. Request and identity	Authenticate the user, capture intent and constraints, classify risk, and establish tenant and data boundaries.
2. Policy and orchestration	Select a workflow or agent policy, create budgets and checkpoints, and decide whether planning, retrieval, tools, or human approval are required.
3. Model and context layer	Construct trusted instructions, retrieve relevant memory or knowledge, call the appropriate model, and validate structured outputs.
4. Tool and execution layer	Apply least privilege, semantic validation, idempotency, sandboxing, rate limits, and post-condition checks to every action.
5. Evidence and observability	Persist state transitions, tool receipts, approvals, provenance, metrics, and evaluation signals for replay and audit.

SECTION 01

Agent Architecture and Design Decisions

These questions test whether the candidate can choose the right control model, separate deterministic and probabilistic responsibilities, and design an agent as a production system rather than a prompt loop.

Section focus: Start with autonomy and risk, then choose the simplest control structure that can satisfy the task. Make state, trust boundaries, tool contracts, and recovery behavior explicit.

1. Design an AI agent that receives a user goal, creates a plan, executes tools, validates results, and produces a final answer. What components would you separate, and what state would each component maintain?

The interviewer expects an end-to-end architecture, clear component boundaries, and an explanation of how execution remains recoverable and auditable.

Model answer

I would put a deterministic orchestrator around the model. The orchestrator owns the run identifier, user and tenant identity, budgets, current state, policy decisions, retries, and termination. The model may propose a plan or next action, but it does not own durable state or directly invoke privileged services. This keeps probabilistic reasoning replaceable and prevents the language model from becoming the control plane.

The request enters through an API and identity layer, then passes through risk classification and policy checks. A planner produces a structured task graph with dependencies and success criteria. An executor selects tools through a governed tool gateway. Every result is written to an evidence store and checked by a validator before the state machine advances. A response composer uses only approved evidence and execution receipts. For long tasks, the workflow engine checkpoints after every state transition so a worker can restart without replaying completed side effects.

Architecture and implementation

- Request and policy layer: authenticate the caller, normalize the goal, classify risk, create a budget, and determine whether approval is required.
- Planner: return a typed plan containing step IDs, dependencies, required capabilities, expected outputs, and completion criteria rather than free-form prose only.
- Executor and tool gateway: enforce schema validation, semantic validation, least privilege, timeouts, rate limits, idempotency keys, and post-condition checks.
- State and evidence stores: persist the plan, step status, observations, tool receipts, artifacts, approvals, and provenance separately from the model context.
- Validator and responder: verify task-level invariants, detect unsupported claims, summarize partial completion, and generate the final answer from validated state.

Bottlenecks, risks, and safeguards

- A single prompt that both plans and acts is difficult to inspect and can drift. Separate planning, execution, and validation with typed contracts.
- Ambiguous tool failures can create duplicate side effects. Use operation IDs, idempotency keys, and status lookup before retrying.
- Context growth can hide earlier constraints. Keep canonical state outside the prompt and build a bounded working context for each step.

A validator implemented with the same model and prompt may repeat the same error. Use deterministic checks where possible and independent evidence or a different evaluator for critical claims.

Validation and operating metrics

End-to-end task success and completion by failure category.

Plan validity, tool selection accuracy, post-condition pass rate, and unsupported-claim rate.

p50 and p95 latency, model tokens, tool calls, retries, and cost per successful task.

Recovery success, duplicate-side-effect rate, approval compliance, and audit-trace completeness.

What the interviewer is testing: Whether you can design a bounded, stateful, auditable agent and explain why the orchestrator - not the LLM - owns execution control.

2. A business workflow has twelve well-defined steps with only two requiring judgment. Would you use a fully autonomous agent, a deterministic workflow, or a hybrid design? Explain your decision.

The key is resisting unnecessary autonomy and identifying the smallest area in which an LLM adds value.

Model answer

I would choose a hybrid workflow. The ten well-defined steps should be encoded as deterministic states with explicit inputs, outputs, validations, and retry behavior. I would insert bounded model tasks only at the two judgment points, and require those tasks to return structured decisions with confidence, evidence, and an allowed action class. This gives the business predictable execution while preserving flexibility where rules are insufficient.

The workflow engine remains the source of truth. It decides when to call the model, validates the returned schema, enforces policy thresholds, and selects the next deterministic transition. High-impact or low-confidence judgments can be routed to a person. This architecture is easier to test than a free-form agent and reduces token cost, latency, and failure paths.

Architecture and implementation

Represent the twelve steps as a versioned state machine or workflow definition with explicit preconditions and postconditions.

Define each LLM judgment as a narrow classification, extraction, ranking, or recommendation contract rather than an open-ended goal.

Use rules for hard policy boundaries and the model only for interpretation within those boundaries.

Persist the model input, output, evidence, confidence, and selected branch for audit and offline evaluation.

Provide a human-review branch for novel cases, policy conflicts, or decisions above a materiality threshold.

Bottlenecks, risks, and safeguards

A fully autonomous agent creates many unnecessary choices and makes path coverage difficult.

A purely deterministic workflow may become a brittle tree of exceptions. Keep the judgment interfaces narrow but replaceable.

Model confidence is not calibrated by default. Calibrate against labeled cases and combine it with rule-based risk signals.

Workflow changes and prompt changes can become inconsistent. Version the complete decision package and regression-test it together.

Validation and operating metrics

- Accuracy and calibration of the two judgment steps.
- Straight-through processing rate, human-review rate, and reversal rate after review.
- Workflow completion, branch-level failure rate, latency, and cost compared with a full-agent baseline.
- Policy violation rate and reproducibility across equivalent inputs.

What the interviewer is testing: Whether you apply the principle of minimum necessary autonomy and can combine workflow reliability with bounded model judgment.

3. How would you decide whether a use case needs an agent at all, rather than a standard LLM call, RAG pipeline, rules engine, or traditional application?

This tests problem formulation and whether the candidate treats agents as one option rather than the default.

Model answer

I would classify the use case along four axes: uncertainty in the required path, need for external actions, duration and statefulness, and consequence of a wrong action. A single LLM call is appropriate when the task is one-shot transformation or generation. RAG is appropriate when the main uncertainty is factual context. Rules or conventional code are appropriate when inputs and outcomes are well specified. An agent is justified when the system must choose among tools, adapt the sequence based on observations, maintain state, and pursue a goal across multiple steps.

I would then run a staged proof. First build the simplest non-agent baseline. Measure its coverage and identify the unresolved cases. Add a deterministic workflow for known branches. Only introduce an agent for the residual portion where dynamic planning creates measurable value. This avoids paying an autonomy tax in reliability, latency, security, and operational complexity without evidence.

Architecture and implementation

- Create a decision matrix covering task variability, number of systems involved, reversibility, required memory, latency, and regulatory impact.
- Prototype a rules, RAG, or workflow baseline and compare it with an agent on the same task set.
- Define an autonomy envelope: permitted goals, tools, budgets, data domains, and actions.
- Prefer a router that sends simple cases to cheaper deterministic paths and only escalates complex cases to an agent.
- Require a measurable business objective such as increased resolution rate or reduced handling time, not novelty alone.

Bottlenecks, risks, and safeguards

- Agent enthusiasm can hide that the problem is primarily search, data quality, or integration.
- An agent can make a deterministic process less reproducible and harder to certify.
- A benchmark that contains only complex cases may overstate the need for an agent. Evaluate the real traffic distribution.
- Adding tools increases the security surface. Each capability must justify its risk and maintenance cost.

Validation and operating metrics

- Incremental task-success lift over the simplest viable baseline.
- Coverage of dynamic cases, path length, manual effort saved, and time to completion.
- Cost, latency, failure recovery, and safety incidents per completed task.
- Percentage of traffic that truly needs agentic execution after routing.

What the interviewer is testing: Whether you can diagnose the actual source of complexity and justify agentic behavior with evidence rather than fashion.

4. Design an agent architecture that supports multiple model providers without tightly coupling agent logic to a specific model API.

The expected answer covers a model gateway, normalized contracts, capability routing, fallbacks, observability, and provider-specific escape hatches.

Model answer

I would introduce a model gateway with a provider-neutral request and response contract. Agent code asks for a capability such as structured planning, tool selection, summarization, or high-accuracy reasoning, along with constraints for latency, cost, region, context length, and data handling. The gateway resolves that request to a model deployment and translates tool schemas, structured-output settings, streaming events, token accounting, and errors into a common internal representation.

The abstraction should normalize the stable 80 percent, not erase meaningful differences. Provider-specific features can be exposed through declared capability flags or optional extensions. Prompts, model policies, and evaluation thresholds are versioned outside application code. A routing policy can choose a preferred model, perform canarying, and fall back only when the alternate model is validated for that task.

Architecture and implementation

- Define internal types for messages, content parts, tool calls, usage, safety results, finish reasons, and typed errors.
- Maintain a capability registry for context length, modalities, structured output, tool support, data residency, latency tier, and cost.
- Use task-based routing rather than hard-coded provider names inside agent nodes.
- Add circuit breakers, health checks, quotas, retry budgets, and provider-specific rate-limit handling in the gateway.
- Record model, provider, deployment, prompt version, parameters, and normalized response in every trace.

Bottlenecks, risks, and safeguards

- A lowest-common-denominator interface can block useful features. Support optional capabilities and fail fast when a route cannot satisfy the contract.
- Silent fallback can change quality or compliance. Fallbacks should be task-approved, observable, and region-aware.
- Equivalent temperature or tool semantics are not guaranteed across providers. Validate behavior, not parameter names.
- Provider outages can trigger a thundering herd against the fallback. Apply capacity-aware load shedding and admission control.

Validation and operating metrics

- Success, latency, cost, and safety metrics by task, model, provider, and route.
- Fallback rate, failover success, routing regret, and circuit-breaker activity.
- Structured-output validity and tool-call correctness across providers.
- Regression results and canary deltas before changing a route.

What the interviewer is testing: Whether you can create portability without pretending models are interchangeable, and whether routing is governed by measured capabilities.

5. Which parts of an agent system should be deterministic, and which parts should be model-driven?

A strong response draws a trust boundary and assigns probabilistic reasoning only where semantic ambiguity creates value.

Model answer

I use deterministic code for identity, authorization, budgets, state transitions, input and output schemas, policy enforcement, arithmetic, database writes, idempotency, retries, timeouts, and final confirmation of side effects. These are areas where consistency and auditability matter more than linguistic flexibility. I use models for intent interpretation, ambiguous classification, information extraction from unstructured content, planning under incomplete information, ranking, summarization, and drafting.

The boundary is not static. A model may propose a plan, SQL query, refund category, or code patch, but deterministic services validate and execute it. When a model decision becomes frequent and well understood, I try to distill it into rules or a smaller classifier. When rules accumulate exceptions, I may reintroduce a bounded model decision. The design should make this substitution easy.

Architecture and implementation

- List all decisions and classify them by ambiguity, consequence, reversibility, and verification cost.
- Use typed model outputs so every model decision enters deterministic validation before changing state.
- Place hard policy constraints in a policy engine rather than relying on prompt wording.
- Use deterministic tools for calculations and source-of-truth lookups, with the model coordinating rather than inventing values.
- Add human approval where the outcome is material and cannot be reliably validated after execution.

Bottlenecks, risks, and safeguards

- Putting rules only in prompts makes them vulnerable to prompt injection and model variation.
- Over-deterministic designs can become unmaintainable exception trees. Track rule complexity and unresolved-case rates.
- A typed schema proves syntax, not correctness. Add semantic checks and domain invariants.
- Using an LLM as the sole verifier of another LLM can create correlated failures.

Validation and operating metrics

- Decision accuracy and calibration for model-driven components.
- Invariant violations and authorization denials for deterministic controls.
- Human override, exception, and fallback rates by decision type.
- Reproducibility, cost, and latency for equivalent tasks.

What the interviewer is testing: Whether you understand that reliable agents are hybrid systems and can justify each probabilistic decision boundary.

6. How would you structure the control loop for an agent that repeatedly observes, reasons, acts, and validates?

The interviewer is looking for an explicit state machine, bounded context construction, and safe termination rather than an unconstrained while loop.

Model answer

I would implement the loop as explicit states: ingest, assess, plan or select-next-step, authorize, act, observe, validate, update state, and either continue, complete, pause, or fail. Each transition has a typed input, a guard condition, a durable checkpoint, and a bounded retry policy. The model produces proposals inside selected states, while the orchestrator determines transitions.

Before every iteration, a context builder retrieves the current goal, immutable constraints, concise execution history, relevant evidence, and available tool subset. After a tool call, the observation is normalized and stored before validation. The validator compares the result with step-level success criteria and task-level invariants. A progress monitor detects repeated states, no-op actions, and budget exhaustion. Completion requires explicit evidence that the goal has been met, not merely a model statement that it is done.

Architecture and implementation

- Represent the run as an append-only event log plus a materialized current-state record.
- Give every action a unique operation ID and store intent, arguments, authorization decision, response, and post-condition.
- Build context per state from canonical data rather than appending the entire transcript indefinitely.
- Use distinct validation paths for content quality, tool success, policy compliance, and overall goal completion.
- Define terminal states for success, partial completion, awaiting approval, awaiting user input, exhausted budget, and unrecoverable failure.

Bottlenecks, risks, and safeguards

- Unbounded reflection loops waste cost and can degrade correct decisions. Limit reflection to detected uncertainty or failed validation.
- Retrying the model with the same context often repeats the error. Change the strategy, tool, evidence, or escalation path.
- A tool may return success before downstream consistency is achieved. Validate observable post-conditions when possible.
- A large transcript can bury instructions. Keep immutable constraints in a protected state field and re-inject them deliberately.

Validation and operating metrics

- Iterations and tool calls per successful task, plus distribution of terminal states.
- No-progress detections, loop interventions, retry effectiveness, and budget exhaustion.
- Step validation pass rate and false-completion rate.
- Checkpoint recovery time and state-replay correctness.

What the interviewer is testing: Whether you can transform the observe-think-act concept into an engineered workflow with guards, durable state, and measurable termination.

7. Compare ReAct-style agents, plan-and-execute agents, state-machine agents, graph-based workflows, and event-driven agents. When would you use each?

The answer should compare control, flexibility, observability, and operational fit rather than declaring one pattern universally best.

Model answer

ReAct is useful for short exploratory tasks where the next action depends directly on the latest observation and the action space is small. Plan-and-execute is better when a task benefits from global decomposition, parallel work, or cost estimates before acting. A state machine is best for regulated or repeatable processes with known transitions. A graph workflow supports branching, joins, cycles, and reusable nodes while keeping explicit control. Event-driven agents fit long-running work in which external events, asynchronous tools, or multiple services advance the process over time.

In production I often combine patterns. A graph or workflow engine provides durable orchestration, while a ReAct-like node handles a bounded research subtask. A planner can create or parameterize a graph, but only approved node types can execute. The choice depends on path variability, task duration, failure cost, concurrency, and how much behavior must be inspectable before execution.

Architecture and implementation

- Use ReAct for bounded investigation with read-only tools, low consequence, and strong step limits.
- Use plan-and-execute when dependencies, parallelism, and progress against an explicit plan matter.
- Use state machines for transactional workflows, approvals, SLAs, and strict audit requirements.
- Use graph workflows when the process has conditional branches, fan-out and fan-in, subgraphs, or controlled loops.
- Use event-driven orchestration for hours-long or days-long tasks, callbacks, queues, and human or system events.

Bottlenecks, risks, and safeguards

- ReAct can wander and is hard to predict under a large tool set.
- Upfront plans can become stale and should support bounded replanning.
- State machines can explode into many transitions if model judgment is not encapsulated.
- Event-driven systems add ordering, deduplication, and eventual-consistency challenges.

Validation and operating metrics

- Task success, path length, replanning frequency, and branch-level failure by pattern.
- Recovery time, replayability, and percentage of runs requiring manual intervention.
- Latency and cost under short, long, and parallel workloads.
- Change effort when adding a new branch, tool, or policy.

What the interviewer is testing: Whether you understand agent patterns as control structures with different operational properties and can compose them pragmatically.

8. An agent must support both short interactive tasks and long-running tasks lasting several hours. Would you use the same architecture for both? Why or why not?

This tests separation of synchronous request handling from durable asynchronous orchestration.

Model answer

I would expose a common task contract and shared platform services, but use different execution modes. Interactive tasks can run through a low-latency synchronous path with tight step and token budgets, streaming progress, and aggressive timeouts. Long-running tasks should be submitted to a durable workflow engine or queue, return a task ID immediately, checkpoint every transition, and support pause, cancellation, approval, and resumption.

Both modes should share identity, policy, model gateway, tool registry, state schema, evidence storage, and observability. The difference is lifecycle management. A web request should not hold process memory or a network connection for hours. Long runs need leases, heartbeats, deduplication, event handling, and worker replacement. A task can migrate from the interactive path to the asynchronous path when predicted complexity or elapsed time exceeds a threshold.

Architecture and implementation

- Add an admission controller that estimates complexity and chooses synchronous, deferred, or human-assisted execution.
- Use a workflow engine with durable timers, retries, signals, and versioned state for long tasks.
- Provide progress events and a consistent task-status API for both modes.
- Separate ephemeral working context from durable canonical state and artifacts.
- Reserve capacity and apply distinct quotas so long jobs cannot starve interactive traffic.

Bottlenecks, risks, and safeguards

- Running long work in request handlers causes timeout, process-loss, and retry duplication.
- Long tasks can continue after the user no longer wants them. Support cancellation tokens and check them at safe points.
- Workflow code changes can break replay. Version workflow definitions and use compatible state migrations.
- Frequent progress generation can become expensive. Emit state-derived progress rather than an extra LLM narration at every step.

Validation and operating metrics

- Interactive p95 latency, abandonment, and first-token time.
- Long-task completion time, checkpoint frequency, recovery success, and cancellation latency.
- Queue delay, worker utilization, lease expirations, and duplicate action rate.
- Percentage of tasks correctly routed to each execution mode.

What the interviewer is testing: Whether you can distinguish a chat request from a durable job while preserving a coherent platform and user experience.

9. How would you design an agent so that planning, tool execution, validation, and response generation can be independently replaced or upgraded?

The expected discussion is about typed interfaces, versioning, event contracts, dependency injection, and evaluation gates.

Model answer

I would define each capability as a component behind a typed contract and keep canonical state independent of any model prompt. The planner consumes a task specification and capability catalog and emits a plan schema. The executor consumes one approved step and returns a normalized result envelope. Validators consume state and evidence and emit findings. The response generator consumes only validated artifacts and a response policy. Components communicate through versioned data models or events, not shared prompt text.

Each implementation is selected through configuration or a registry. For example, a task may use a rules planner, a small model planner, or a high-reasoning planner without changing the executor. Every component version is recorded in the run. Shadow evaluation and canary routing allow a new component to run against production-like traffic before it controls outcomes.

Architecture and implementation

- Create stable domain schemas for task, plan, step, action request, action result, validation finding, and final response.
- Use adapters at provider boundaries and keep provider payloads out of core state.
- Version prompts, policies, schemas, and component implementations independently but test approved bundles.
- Build a replay harness that can rerun one component against stored inputs without re-executing side effects.
- Support feature flags, shadow mode, canaries, and immediate rollback by component version.

Bottlenecks, risks, and safeguards

- Overly generic contracts can lose important semantics. Keep domain-specific extension fields with explicit ownership.
- Independent upgrades can create incompatible combinations. Maintain compatibility matrices and integration regression suites.
- Replaying tool execution can be unsafe. Replay recorded outputs or use isolated test doubles.
- A component can pass local tests but reduce end-to-end quality. Gate on both component and task-level metrics.

Validation and operating metrics

- Schema compatibility and contract-test pass rate.
- End-to-end quality delta by component version and bundle.
- Rollback time, canary incident rate, and shadow disagreement rate.
- Engineering lead time to add or replace a planner, model, validator, or tool provider.

What the interviewer is testing: Whether you can build an evolvable agent platform with explicit contracts rather than a monolithic chain of prompts.

10. An agent handles customer support, refunds, account updates, and policy questions. How would you divide the system into specialized capabilities without creating excessive orchestration complexity?

This tests domain decomposition, routing, shared services, and the difference between capability boundaries and a proliferation of conversational agents.

Model answer

I would use one front-door assistant and a small set of capability workflows rather than a separate free-form agent for every intent. The front door authenticates the customer, detects intent and risk, and routes to modules such as knowledge answer, account lookup, account mutation, refund decision, and escalation. Each module has a narrow tool set, domain policy, state schema, and success criteria. Shared services provide identity, policy evaluation, retrieval, tool execution, approvals, and audit logging.

Policy questions may use RAG and answer generation. Account changes should use deterministic workflows with explicit confirmation. Refunds should combine policy rules, structured evidence extraction, and approval thresholds. The orchestration graph remains shallow: classify, route, execute one primary workflow, and optionally escalate. Cross-domain cases use a case object that allows modules to contribute artifacts without giving every module the full conversation and all tools.

Architecture and implementation

- Define capabilities by distinct authorization and transaction boundaries, not merely by topic labels.
- Use hierarchical routing: coarse intent and risk first, then module-specific decisions inside the selected workflow.
- Keep customer and case state in shared canonical stores while giving each module a least-privilege view.
- Standardize action receipts, error envelopes, escalation reasons, and response evidence across modules.
- Use a common policy engine so refund and account workflows do not implement conflicting versions of policy.

Bottlenecks, risks, and safeguards

- Too many agents create routing ambiguity, context duplication, and hard-to-debug handoffs.
- A single all-powerful agent violates least privilege and increases blast radius.
- Intent classification errors can trigger the wrong authority. Recheck authorization at the tool gateway, not only at routing.
- Policy retrieval may return obsolete documents. Rank by effective date and source authority and expose conflicts.

Validation and operating metrics

- Intent routing accuracy, module completion, escalation appropriateness, and transfer rate.
- Refund correctness, duplicate refund rate, account-update reversals, and policy citation support.
- End-to-end handle time, tool calls, latency, and cost by capability.
- Unauthorized-action attempts blocked and cross-customer isolation tests.

What the interviewer is testing: Whether you can create bounded domain capabilities with shared governance while avoiding both a monolith and unnecessary multi-agent complexity.

SECTION 02

Planning, Reasoning, and Task Decomposition

These questions examine how an agent converts broad goals into executable work, revises plans safely, preserves constraints, and avoids expensive or unproductive reasoning loops.

Section focus: Plans should be typed, feasible, dependency-aware, and linked to measurable success criteria. Replanning should occur because state changed or validation failed, not because the model feels uncertain.

11. An agent receives the goal, "Improve our customer retention." How should it transform this broad request into actionable subtasks?

The challenge is converting an underspecified business objective into a bounded analytical and execution plan without inventing authority.

Model answer

The agent should not immediately generate campaigns or change customer accounts. It should first clarify the objective: target segment, baseline retention definition, time horizon, acceptable interventions, budget, available data, and decision owner. If interaction is possible, it asks focused questions. Otherwise it records assumptions and limits the first phase to analysis and recommendations.

I would decompose the work into discovery, diagnosis, option generation, experiment design, and approval. Discovery identifies data sources and constraints. Diagnosis quantifies churn by cohort, journey stage, and controllable drivers. Option generation maps evidence to candidate interventions. Experiment design defines treatment, control, guardrails, sample size assumptions, and success metrics. Execution remains behind explicit approval because the original goal did not authorize customer-facing changes.

Architecture and implementation

- Translate the goal into a task specification containing objective, scope, actors, constraints, authority, deliverables, and completion criteria.
- Build a dependency graph: data access and metric definition precede diagnosis; diagnosis precedes intervention selection; approval precedes action.
- Use domain tools for cohort analysis and causal or experimental reasoning rather than asking the LLM to infer drivers from summaries alone.
- Attach evidence requirements to every recommendation, including source data, uncertainty, expected impact, cost, and possible harm.
- Produce staged deliverables so stakeholders can approve the problem definition before expensive analysis or external action.

Bottlenecks, risks, and safeguards

- The goal is ambiguous and could cause scope expansion. Enforce a discovery state and an explicit autonomy boundary.
- Correlation may be presented as causation. Require experiments or credible causal evidence before claiming an intervention will improve retention.
- Optimizing retention can harm margin or customer trust. Include guardrails such as complaints, discount cost, and long-term value.
- The agent may use sensitive attributes improperly. Apply data-governance policy and fairness review to segmentation.

Validation and operating metrics

- Completeness and correctness of the task specification and dependency graph.
- Evidence coverage and uncertainty calibration for proposed retention drivers.
- Stakeholder approval rate, rework caused by missing assumptions, and time to an experiment-ready recommendation.
- For approved experiments: retention lift, incremental value, guardrail movement, and statistical or practical significance.

What the interviewer is testing: Whether you can turn a vague goal into a safe, evidence-driven program while distinguishing analysis authority from execution authority.

12. An agent produces a reasonable plan but skips several steps during execution. How would you ensure that execution remains aligned with the approved plan?

This tests the distinction between a plan as prose and a plan as executable state.

Model answer

I would represent the plan as a structured task graph rather than a text block. Every step has an identifier, dependencies, status, required inputs, expected outputs, validation rule, and allowed tools. The orchestrator selects only steps whose dependencies are complete and prevents the model from marking a step complete without an artifact or validation result.

The model can propose the next step, but the workflow engine reconciles that proposal with the canonical graph. If the model requests a later step, the engine returns the missing prerequisites. When new information requires a change, the agent creates a plan revision with a reason and impact analysis; it does not silently abandon approved work. The final response compares planned, completed, skipped, and added steps.

Architecture and implementation

- Persist plan nodes and edges in durable state with explicit lifecycle values such as pending, ready, running, blocked, completed, failed, and waived.
- Define completion evidence for each step, for example a tool receipt, document, test result, or validated structured record.
- Use guards that refuse transitions when dependencies, approvals, or budgets are missing.
- Version plan revisions and require policy or human approval when a revision changes scope, cost, risk, or irreversible actions.
- Add a reconciliation pass before finalization to detect uncompleted mandatory steps and unresolved findings.

Bottlenecks, risks, and safeguards

- A model may produce plausible status narration without execution. Derive status from state and receipts, not generated text.
- Overly rigid plans fail when observations change. Allow controlled replanning with explicit diffs.
- Parallel branches can finish out of order. Use dependency and join semantics instead of sequence assumptions.
- A waived step can become a hidden bypass. Record who or what waived it, why, and the remaining risk.

Validation and operating metrics

- Mandatory-step completion and dependency-violation rate.
- Unapproved plan-deviation rate and number of revisions per task.
- False-completion findings during reconciliation.

Execution efficiency relative to the critical path and approved budget.

What the interviewer is testing: Whether you can make plans operationally enforceable and auditable rather than relying on the model to remember its own prose.

13. How would you evaluate whether an agent's task decomposition is too coarse, too granular, or appropriately scoped?

A good response defines practical properties of a useful step and explains how to evaluate decomposition offline and online.

Model answer

A useful step should have one clear objective, a bounded set of inputs, an observable output, an assigned capability, and a validation rule. A decomposition is too coarse when a step requires many unrelated tools, has several failure modes that cannot be localized, or produces an output that is difficult to verify. It is too granular when orchestration overhead, context switching, and repeated model calls dominate the actual work or when steps have no independently meaningful output.

I would evaluate decompositions against a labeled task set and execution traces. Compare success, retries, tool calls, latency, cost, and fault localization. I would also inspect whether steps align with transaction and trust boundaries. A payment proposal and payment execution should be separate even if combining them is cheaper, because they require different authorization and rollback behavior.

Architecture and implementation

- Score each step for cohesion, verifiability, tool feasibility, side-effect boundary, and context requirement.
- Prefer steps that can be retried or replaced independently and that produce reusable artifacts.
- Group small deterministic transformations into code rather than separate LLM steps.
- Split high-risk actions from analysis and place approval immediately before the exact side effect.
- Use hierarchical plans: stable milestones at the top and detailed subplans generated only when a milestone becomes active.

Bottlenecks, risks, and safeguards

- Granular plans create excessive tokens, persistence writes, and failure points.
- Coarse steps hide partial failure and make resumption unsafe.
- The same granularity does not fit all tools; long-running APIs and local calculations have different boundaries.
- A decomposition evaluator may favor short paths even when extra validation is necessary. Include risk-adjusted path quality.

Validation and operating metrics

- Task success, average step count, retries, and cost per successful task.
- Percentage of failures localized to a single step and resumption success after failure.
- Validation coverage and side-effect isolation.
- Critical-path duration versus orchestration overhead.

What the interviewer is testing: Whether you can reason about decomposition as a systems-design decision balancing cohesion, verification, recoverability, and overhead.

14. An agent creates a plan containing steps that cannot be completed using its available tools. How would you detect and repair such plans before execution?

This tests capability grounding and feasibility checking before expensive or risky execution begins.

Model answer

The planner should receive a capability catalog containing tool purpose, required inputs, permissions, side effects, cost, latency, and known constraints. It must reference capability IDs in each plan step. A deterministic plan validator then checks that every step maps to an available capability, that required data and credentials can be obtained, and that the caller is authorized to use it.

If a step is infeasible, the system should classify why: missing tool, missing input, missing permission, unsupported region, budget conflict, or unverifiable output. The repair policy can substitute an approved capability, narrow the goal, ask the user for information, route to a person, or terminate with a transparent limitation. The agent should not invent a tool or imply that an unavailable action occurred.

Architecture and implementation

- Use machine-readable tool manifests and capability tags instead of relying only on natural-language descriptions.
- Perform static plan checks for capability coverage, dependency satisfiability, estimated budget, and permission requirements.
- Run lightweight dry-run or validation endpoints for high-risk integrations before committing to a plan.
- Allow the planner to request capability discovery from a curated registry rather than seeing hundreds of full schemas.
- Persist infeasibility reasons so planners and tool descriptions can be improved from real failures.

Bottlenecks, risks, and safeguards

- Tool descriptions may be stale or overly broad. Version manifests and test them against live contracts.
- A nominally available tool may be unhealthy or rate limited. Include current health and capacity in feasibility.
- Automatic substitution can alter semantics. Require equivalence rules and user or policy approval for material changes.
- The model may claim it can solve the step manually. The orchestrator should require a declared output method and evidence.

Validation and operating metrics

- Plan feasibility pass rate and execution failures caused by unavailable capabilities.
- Repair success by infeasibility category.
- Tool-manifest accuracy and stale-capability incidents.
- Time and model calls spent before detecting an impossible plan.

What the interviewer is testing: Whether you ground planning in real capabilities and can fail or repair explicitly instead of hallucinating execution power.

15. Should an agent generate its entire plan upfront or plan incrementally after each observation? Discuss the trade-offs.

The best answer usually proposes a hybrid with stable milestones and adaptive local planning.

Model answer

Upfront planning provides global coherence, dependency visibility, cost estimates, approval opportunities, and parallelism. It works well when the domain is known and observations are predictable. Incremental planning adapts better to uncertain environments, avoids spending effort on branches that may not be needed, and uses fresh information. Its weaknesses are myopic choices, repeated replanning, and weaker cost predictability.

I normally use hierarchical planning. The agent creates a high-level plan with milestones, budgets, risk classes, and completion criteria. It then generates a detailed subplan only for the active milestone. After each material observation, it checks whether assumptions or dependencies changed. Minor changes stay local; major changes create a versioned revision to the high-level plan and may require approval.

Architecture and implementation

- Define what is stable enough to plan globally: milestones, irreversible actions, approvals, budgets, and dependencies.
- Define replanning triggers such as failed validation, changed source data, unavailable tools, or newly discovered constraints.
- Estimate uncertainty and use more incremental planning in exploratory tasks and more upfront planning in transactional tasks.
- Reserve resources for likely parallel branches but avoid executing speculative side effects.
- Compare the current state with plan assumptions before each milestone transition.

Bottlenecks, risks, and safeguards

- A detailed upfront plan can become obsolete and create sunk-cost behavior.
- Purely incremental agents can wander without a global completion model.
- Frequent replanning can consume more tokens than execution. Use deterministic triggers and a revision budget.
- A plan revision may invalidate earlier approvals. Bind approvals to plan version and action digest.

Validation and operating metrics

- Plan stability, replanning frequency, and percentage of revisions that improve task outcome.
- Budget prediction error and unused speculative work.
- Path efficiency and time lost to stale plans.
- Approval invalidation and reapproval frequency after material revisions.

What the interviewer is testing: Whether you can balance global structure with adaptation and define concrete conditions for replanning.

16. How would you prevent an agent from repeatedly modifying its plan without making meaningful progress?

The expected response covers progress signals, revision limits, similarity detection, and escalation.

Model answer

I would require every plan revision to reference a new observation, failed validation, or changed constraint. The revision must state what changed, which steps are affected, and why the new plan is expected to improve the outcome. The orchestrator computes a plan diff and rejects cosmetic changes or revisions that repeat a recently failed strategy without new evidence.

A progress monitor tracks completed artifacts, reduction in unresolved requirements, state transitions, and distance to success criteria. It also detects repeated plans, repeated tool arguments, and cycles between equivalent states. After a bounded number of ineffective revisions, the system changes strategy, asks for user input, delegates to a specialist, or terminates with a partial result.

Architecture and implementation

- Store plan fingerprints and semantic summaries to identify near-duplicate revisions.
- Assign a revision budget and a minimum expected-value threshold for another planning call.
- Require revision reasons from a controlled taxonomy and link them to evidence.
- Track monotonic progress variables such as number of verified requirements, unresolved blockers, or completed dependencies.
- Escalate to a different model, tool, or human only when the failure category supports that change.

Bottlenecks, risks, and safeguards

- String comparison misses semantically identical plans. Combine structural and embedding-based similarity.
- Overly strict progress rules can stop legitimate exploration. Use domain-specific milestones and uncertainty bounds.
- Switching models without changing evidence may reproduce the same behavior at higher cost.
- The agent may create trivial artifacts to appear productive. Validate artifact relevance and contribution to success criteria.

Validation and operating metrics

- Revisions per task, duplicate-plan rate, and no-progress termination rate.
- Incremental success probability after each revision.
- Cost and latency spent on planning versus execution.
- Recovery success after strategy change or escalation.

What the interviewer is testing: Whether you can operationalize progress and stop costly meta-reasoning loops rather than relying on a fixed iteration cap alone.

17. An agent discovers new information that invalidates its original plan. How should it decide whether to replan, continue, ask the user, or stop?

This tests change-impact analysis, authority boundaries, and safe handling of invalid assumptions.

Model answer

The agent should compare the new information with explicit plan assumptions and classify the impact. If it affects only a local implementation detail and stays within scope, budget, and authority, it can repair the active step. If it changes dependencies or the expected path but not the objective, it should create a plan

revision. If it changes user intent, cost, timeline, data use, risk, or an approved action, it should pause and ask the user or approver. If the objective is no longer achievable, illegal, unsafe, or unsupported, it should stop and explain what was completed and why continuation is not valid.

This decision belongs in policy and orchestration, not free-form model intuition. The model can classify and explain the impact, while deterministic rules enforce thresholds for material changes and irreversible actions.

Architecture and implementation

- Maintain an assumption register with source, confidence, affected steps, and materiality.
- Implement an impact matrix for local repair, automatic replan, mandatory approval, and terminal stop.
- Invalidate downstream artifacts that depend on a changed assumption and mark them for recomputation.
- Preserve completed side effects and determine whether compensating actions are required.
- Show the user a concise diff: old assumption, new evidence, affected outcome, proposed next action, and additional cost or risk.

Bottlenecks, risks, and safeguards

- Continuing with invalid assumptions produces internally coherent but wrong results.
- Unnecessary user questions create friction. Ask only when the change crosses an authority or materiality boundary.
- Replanning may reuse invalid derived artifacts. Track dependency lineage and invalidate transitively.
- Stopping after partial side effects can leave inconsistent external state. Trigger compensation or escalation.

Validation and operating metrics

- Accuracy of impact classification and user agreement with escalation decisions.
- Stale-artifact reuse incidents and dependency invalidation coverage.
- Replan success, abandoned-task rate, and cost of unnecessary replanning.
- Consistency of external state after stop or compensation.

What the interviewer is testing: Whether you can connect changing evidence to scope, approvals, dependency lineage, and safe continuation decisions.

18. How would you design a planner for tasks with dependencies, optional branches, parallelizable steps, and irreversible actions?

The response should look like workflow or DAG design with explicit transaction and approval boundaries.

Model answer

I would represent the plan as a directed graph with typed nodes and edges. Nodes declare required inputs, produced artifacts, capability, estimated cost and duration, risk class, and compensability. Edges represent hard dependencies, conditional branches, or join requirements. The scheduler executes ready, independent, read-only nodes in parallel while respecting quotas and shared-resource constraints.

Irreversible actions are modeled as commit nodes. All prerequisite analysis, validation, authorization, and user confirmation occurs before the commit. The approval is bound to an exact action digest and expires if inputs change. Optional branches have predicates and explicit skip outcomes. Join nodes specify whether all, any, or a quorum of branches must succeed. The planner can propose the graph, but a deterministic validator checks acyclicity where required, reachability, missing joins, and unsafe commit placement.

Architecture and implementation

- Use node types such as read, compute, model decision, approval, side effect, validation, compensation, and human task.
- Annotate resources and concurrency keys to avoid parallel writes to the same entity.
- Add timeout, retry, fallback, and compensation policy per node rather than one global policy.
- Estimate the critical path and total budget before execution; recompute after material changes.
- Persist branch predicates and evidence so the chosen path can be audited.

Bottlenecks, risks, and safeguards

- Parallelism can violate hidden dependencies or API limits. Use explicit resource locks and capacity-aware scheduling.
- Conditional logic generated by a model may be inconsistent. Restrict predicates to a safe expression language or validated functions.
- An irreversible action placed too early increases compensation burden. Enforce pre-commit invariants.
- Graph complexity can exceed the model context. Store it structurally and present only the active neighborhood.

Validation and operating metrics

- Critical-path duration, parallelism efficiency, and queue wait by node type.
- Dependency violations, race conditions, and commit-without-approval incidents.
- Branch decision accuracy and unnecessary branch execution.
- Compensation frequency and recovery success after downstream failure.

What the interviewer is testing: Whether you can combine agent planning with established workflow, concurrency, and transaction-design principles.

19. An agent is solving a complex task but has a limited context window. How would it retain the plan, completed steps, evidence, and unresolved questions?

This tests externalized state, context construction, summarization quality, and provenance.

Model answer

I would not use the conversation transcript as the source of truth. Durable stores hold the typed plan, step states, artifacts, evidence, decisions, assumptions, and unresolved issues. For each model call, a context builder creates a task-specific working set: immutable goal and constraints, the active step, relevant predecessor outputs, selected evidence, and a compact progress summary. Large artifacts remain outside the prompt and are accessed through retrieval or tools.

Summaries are treated as derived caches, not canonical records. Important constraints and decisions are stored as structured fields with provenance. When the agent needs details, it retrieves the original artifact or evidence span. Context selection is driven by dependency links, recency, relevance, and risk, with explicit token budgets per category.

Architecture and implementation

- Use a relational or document state store for canonical task and step records, plus object storage for large artifacts.
- Maintain an evidence index keyed by entities, steps, sources, and claims.
- Generate structured progress summaries containing completed, current, blocked, pending, and unresolved items.
- Pin non-negotiable instructions and user constraints in a protected context segment on every call.
- Record which state and evidence versions were included in each model request for replay.

Bottlenecks, risks, and safeguards

- Recursive summaries can drift. Summarize from canonical events and periodically verify against source records.
- Relevance retrieval may omit a low-similarity but critical constraint. Use dependency and policy-based inclusion in addition to embeddings.
- Duplicated evidence consumes context and biases the model. Deduplicate by source and claim.
- A stale artifact may be retrieved after replanning. Attach version and validity status to every artifact.

Validation and operating metrics

- Context token use by category and task success versus context size.
- Constraint omission, stale-artifact use, and summary-drift rates.
- Evidence retrieval recall for required claims.
- Replay fidelity: whether the recorded context reproduces the decision.

What the interviewer is testing: Whether you understand that context is a generated view over durable state, not the memory system itself.

20. How would you distinguish productive agent reflection from unnecessary self-critique that increases cost and latency?

A strong answer makes reflection conditional, targeted, and measurable rather than an always-on extra prompt.

Model answer

Reflection is productive when it is triggered by a specific uncertainty or failed check and has access to information that can change the decision. Examples include a failed test, contradictory evidence, low-confidence tool selection, or an unsupported claim. It is unproductive when the same model simply rereads the same answer with a generic instruction to improve it. That often adds verbosity, changes correct outputs, and increases correlated error.

I would use targeted validators first. Deterministic checks handle schemas, calculations, permissions, and tests. A critique model receives a narrow rubric and the relevant evidence, identifies concrete defects, and returns structured findings. Revision occurs only for validated findings above a threshold. The system stops when checks pass, the expected value of another pass is low, or the reflection budget is consumed.

Architecture and implementation

- Define reflection triggers and rubrics by task type instead of adding a universal critique step.
- Separate critic and reviser roles and provide the critic with source evidence and success criteria.
- Ask for defect location, severity, evidence, and recommended fix rather than a rewritten answer immediately.
- Use a different model, prompt, tool result, or deterministic checker when independence matters.
- Run reflection selectively based on risk, novelty, uncertainty, and historical error likelihood.

Bottlenecks, risks, and safeguards

- Self-critique can introduce errors into correct output. Preserve the original and require evidence for changes.
- A critic may reward style rather than correctness. Use task-specific rubrics and concise outputs.
- Multiple passes create latency tails. Cap passes and parallelize independent checks where useful.
- Reflection can mask poor upstream retrieval or tools. Route findings to the responsible component.

Validation and operating metrics

- Net quality lift, regression rate, and defect precision of the reflection step.

Additional latency and cost per corrected defect.

Percentage of reflection findings supported by evidence.

Stop-condition accuracy and frequency of unnecessary passes.

What the interviewer is testing: Whether you treat reflection as a targeted verification mechanism with measurable value, not as magical extra reasoning.

SECTION 03

Tool Selection and Tool Execution

These questions focus on capability discovery, safe tool invocation, side-effect control, retries, idempotency, schema evolution, and truthful reporting of execution outcomes.

Section focus: Treat every tool call as an external transaction. Selection, authorization, argument validation, execution, and post-condition verification should be separate, observable stages.

21. An agent has access to 200 tools and frequently selects the wrong one. How would you design tool discovery and routing?

The core issue is overwhelming the model with a flat tool catalog and failing to separate capability routing from argument generation.

Model answer

I would use hierarchical capability routing. First, a deterministic or small-model router classifies the request into a domain and risk class, such as billing lookup, customer mutation, analytics, communication, or code execution. It retrieves a small set of candidate capabilities from a tool registry using semantic descriptions, metadata, permissions, and current health. Only the top candidates and their concise contracts are exposed to the acting model.

Tool selection should be evaluated separately from argument generation. The selector returns a capability ID and justification against required inputs and expected outputs. A policy layer filters candidates by caller permissions, tenant, region, data sensitivity, and action risk. Where possible, common sequences are represented as higher-level workflow tools so the model chooses an intent-level operation instead of coordinating many low-level APIs.

Architecture and implementation

- Maintain a curated registry with purpose, examples, anti-examples, input requirements, output type, side effects, latency, cost, and ownership.
- Use domain and capability retrieval before loading full schemas; cache candidate sets for repeated intents.
- Create aliases or composite tools for common business actions and deprecate overlapping low-level tools.
- Add a deterministic compatibility check between task requirements and selected tool capabilities.
- Capture selection alternatives and confidence for offline error analysis and retraining.

Bottlenecks, risks, and safeguards

- Similar names and descriptions cause confusion. Include discriminating anti-examples and expected post-conditions.
- Tool retrieval can return unauthorized tools. Apply permission filtering before and after model selection.
- The registry can become stale. Require owners, health checks, versioning, and automated contract tests.
- A composite tool can hide risky side effects. Preserve approval and audit boundaries inside the workflow.

Validation and operating metrics

- Top-1 and top-k tool selection accuracy by intent and risk class.
- No-tool-needed accuracy, unauthorized selection attempts, and fallback rate.
- Candidate-set size, selection latency, and schema-token consumption.
- End-to-end task success and recovery after an incorrect initial selection.

What the interviewer is testing: Whether you can scale a tool ecosystem through retrieval, hierarchy, metadata, and governance rather than putting 200 schemas in one prompt.

22. Would you provide all tool schemas directly to the model, retrieve tool descriptions dynamically, or use a separate tool-selection model?

The expected answer compares these options and proposes a context- and risk-dependent combination.

Model answer

Providing all schemas is acceptable only for a small, stable tool set because it gives the acting model complete visibility and avoids a routing stage. At scale it consumes context, increases confusion, and exposes capabilities the user may not be permitted to use. Dynamic retrieval scales better and can incorporate domain, permissions, and health, but retrieval errors can hide the correct tool. A separate selector can be cheaper and more consistent, although it creates another component to train and monitor.

I would combine them: a deterministic policy filter narrows by identity and risk; a lightweight selector or semantic retriever returns a small candidate set; then the acting model receives full schemas for those candidates. For high-risk actions I may route through a fixed workflow rather than tool selection. I would benchmark the whole pipeline against a flat-schema baseline to confirm that routing improves accuracy rather than only reducing tokens.

Architecture and implementation

- Use all schemas for perhaps fewer than ten well-separated tools and short contexts.
- Use registry retrieval for large, evolving catalogs and multi-tenant permission filtering.
- Use a specialized selector when intent labels are stable, volume is high, and latency or cost matters.
- Retain an explicit none-of-the-above outcome and a capability-discovery fallback.
- Version tool descriptions and evaluate retrieval and selection whenever descriptions change.

Bottlenecks, risks, and safeguards

- Dynamic retrieval creates a hidden recall bottleneck. Measure candidate recall before judging the acting model.
- A selector trained on historical traffic may miss new intents. Add novelty detection and human review queues.
- All-schema prompts can leak sensitive capability names or encourage prompt-injection exploration.
- Different models may interpret the same schema differently. Use contract tests per model route.

Validation and operating metrics

- Candidate recall, final selection precision, and task success.
- Prompt tokens, selector cost, end-to-end latency, and catalog growth behavior.
- Unknown-intent detection and manual discovery rate.
- Performance by model, tool version, and permission scope.

What the interviewer is testing: Whether you can articulate the recall, precision, cost, and governance trade-offs in tool exposure.

23. An agent generates syntactically valid tool arguments, but the arguments are semantically incorrect. How would you validate tool calls beyond schema validation?

The challenge is proving that JSON shape is only the first layer and that domain invariants must be enforced outside the model.

Model answer

I would use layered validation. The schema checks type, required fields, ranges, and enumerations. A semantic validator then checks cross-field relationships, identity and ownership, business policy, temporal consistency, and plausibility. For example, a refund amount must not exceed the eligible balance, the currency must match the order, the recipient must be the authenticated account, and the reason code must be supported by evidence.

High-risk calls should use a prepare-validate-commit pattern. The agent submits an action proposal to a domain service, which resolves canonical entities, calculates authoritative values, and returns a human-readable preview plus an action digest. Approval and execution reference that digest. The model should select intent and provide supporting context, while the service derives fields that should never be guessed.

Architecture and implementation

- Resolve user-provided names and identifiers against authoritative records before execution.
- Implement cross-field constraints and domain invariants in code or a policy engine.
- Perform dry-run, quote, or preview calls when the downstream system supports them.
- Use allowlisted transformations and deterministic calculations for amounts, dates, and permissions.
- Validate post-conditions after execution and compare them with the approved intent.

Bottlenecks, risks, and safeguards

- A model-generated identifier may refer to a real but wrong entity. Bind entities to the authenticated session and display them for confirmation.
- Validation services can duplicate business logic and drift. Prefer authoritative domain APIs that expose validation endpoints.
- Plausibility checks alone may approve a harmful but plausible action. Combine policy, identity, and evidence checks.
- Repairing invalid arguments with another model call can silently change intent. Show material changes or require approval.

Validation and operating metrics

- Schema versus semantic rejection rates and top semantic failure categories.
- False-accept and false-reject rates for validation rules.
- Action reversal, customer dispute, and post-condition failure rates.
- Percentage of sensitive fields derived from authoritative systems rather than generated by the model.

What the interviewer is testing: Whether you understand semantic validation, authoritative data derivation, and prepare-commit patterns for safe actions.

24. A tool call times out, but the action may have completed successfully. How should the agent determine whether to retry?

This is the classic ambiguous outcome problem in distributed systems.

Model answer

The agent should not blindly retry a side-effecting operation. Every action request should carry a client-generated operation ID or idempotency key. After a timeout, the executor queries a status or reconciliation endpoint using that ID. If the operation is confirmed complete, it records the receipt and continues. If it is confirmed absent, it may retry under the same key. If status is unknown, the workflow enters an ambiguous state and escalates or waits for asynchronous confirmation according to risk.

The retry policy depends on operation semantics. Read-only calls can usually be retried with bounded exponential backoff. Naturally idempotent writes may be retried. Non-idempotent actions require downstream deduplication or a prepare-commit protocol. The model never decides this from prose; the tool registry declares retry and idempotency behavior.

Architecture and implementation

- Generate an operation ID before the first attempt and persist it with the intended action.
- Require tools to expose idempotency, status lookup, or reconciliation contracts where possible.
- Classify errors into definite failure, definite success, retryable transport failure, and ambiguous outcome.
- Use asynchronous callbacks or event polling for operations whose completion exceeds request timeouts.
- Block dependent steps until the ambiguous operation is resolved or explicitly compensated.

Bottlenecks, risks, and safeguards

- A timeout at the client does not imply failure at the server.
- Changing arguments on retry while reusing an idempotency key can create inconsistent behavior. Bind the key to a request hash.
- Status endpoints may be eventually consistent. Use a bounded reconciliation window and domain-specific escalation.
- Automatic compensation before confirming the original state can create a second error.

Validation and operating metrics

- Ambiguous outcome rate, time to resolution, and unresolved operations by age.
- Duplicate side-effect rate and idempotency conflict rate.
- Retry success by error class and downstream service.
- Percentage of tools with reliable status or reconciliation support.

What the interviewer is testing: Whether you apply distributed transaction principles and avoid equating network failure with business-operation failure.

25. An agent retries a payment request and creates a duplicate charge. How would you design idempotency across the agent, tool gateway, and downstream service?

The answer should describe end-to-end idempotency rather than a retry flag in one layer.

Model answer

The agent creates a stable business-operation ID when the payment intent is approved. That ID is persisted in workflow state and included in every retry. The tool gateway validates that the operation ID is bound to the same customer, amount, currency, payment method, and order. The downstream payment

service stores the idempotency key and returns the original result for repeated identical requests. Conflicting payloads under the same key are rejected.

The workflow separates payment proposal, approval, submission, and reconciliation. It never creates a new key merely because a worker restarted. After submission, it records the provider transaction ID and reconciles with provider status or ledger events. If the provider lacks idempotency, a local transaction coordinator reserves the operation in a database with a unique constraint before the external call, though this still needs careful reconciliation for crashes between call and receipt.

Architecture and implementation

- Use one stable operation ID per approved business intent, not per HTTP attempt.
- Store request hash, workflow ID, actor, approval digest, provider, and result under the operation record.
- Enforce uniqueness at the earliest authoritative write and at the downstream service when available.
- Reconcile provider transactions with the internal ledger before marking the workflow complete.
- Alert on duplicate-looking events and place uncertain cases into a manual financial-operations queue.

Bottlenecks, risks, and safeguards

- Generating a new ID after a timeout defeats deduplication.
- Reusing an ID for modified amount or recipient can conceal a serious error. Reject request-hash mismatches.
- Exactly-once delivery is not guaranteed by queues alone. Design for at-least-once processing with idempotent consumers.
- A duplicate may occur outside the agent path, so reconciliation must operate at the ledger level.

Validation and operating metrics

- Duplicate charge and duplicate submission rates.
- Idempotent replay count, request-hash conflicts, and reconciliation mismatch rate.
- Time from ambiguous submission to confirmed ledger state.
- Manual review volume and customer impact from payment incidents.

What the interviewer is testing: Whether you can carry one business identity through workflow, gateway, provider, and reconciliation layers.

26. How would you classify tools as read-only, reversible, irreversible, privileged, or high-risk, and how should these classifications influence agent behavior?

This tests risk-aware capability governance and enforcement at runtime.

Model answer

I would maintain a risk manifest for each tool and operation, not only for the overall API. Attributes include data sensitivity, read versus write, scope of affected resources, reversibility, financial or legal impact, privilege level, external communication, and whether the result can be verified. The policy engine combines this with caller identity, tenant policy, task context, and environment to determine allowed execution mode.

Read-only, low-sensitivity operations can be automatic with normal budgets. Reversible writes may execute automatically within limits but require post-condition checks and a compensation path. Irreversible or high-impact actions require a prepared preview, exact-action approval, stronger authentication, and potentially segregation of duties. Privileged tools are never exposed directly to the model; the gateway issues narrowly scoped credentials only after authorization.

Architecture and implementation

- Define operation-level risk metadata in the tool registry and review it with security and domain owners.

- Map risk classes to controls: allowed models, approval, rate limits, sandbox, logging, retention, and monitoring.
- Use dynamic risk factors such as amount, recipient domain, data volume, novelty, and time of day.
- Require post-condition verification and compensation for reversible actions.
- Apply stricter controls in production than in isolated test environments, with environment-bound credentials.

Bottlenecks, risks, and safeguards

- A nominally read-only tool can exfiltrate sensitive data. Include confidentiality impact, not just mutation semantics.
- Reversibility may be theoretical or time-limited. Record compensation window and expected residual harm.
- A model can chain low-risk tools into a high-risk outcome. Evaluate cumulative risk at the plan and session level.
- Static labels become stale as APIs gain features. Reassess on schema and permission changes.

Validation and operating metrics

- Tool use and denial rate by risk class.
- Approval compliance, post-condition failures, and compensation success.
- Cumulative-risk policy triggers and attempted privilege escalation.
- Time to update risk metadata after a tool change or incident.

What the interviewer is testing: Whether you can translate tool risk into concrete runtime controls and recognize risk from data access and action chaining.

27. An agent needs to call several independent APIs. How would you decide which calls can run in parallel?

The interviewer expects dependency, consistency, rate-limit, and resource analysis rather than parallelizing everything.

Model answer

I would construct a dependency graph from required inputs and produced outputs. Calls can run in parallel when they do not depend on each other, do not mutate the same resource, do not consume a shared scarce quota in a way that causes failure, and do not require ordered observation for correctness. Read-only retrieval from independent systems is a common candidate. Writes to the same customer, account, branch, or transactional aggregate generally require serialization or optimistic concurrency control.

The scheduler also considers marginal value. It may parallelize multiple evidence sources when each is likely to be needed, but delay expensive speculative calls behind a cheap branch predicate. Results join through an explicit aggregation node with timeout and partial-result semantics. Concurrency is bounded globally, per tenant, per tool, and per resource key.

Architecture and implementation

- Annotate plan steps with dependencies, read and write resource sets, estimated duration, cost, and rate-limit domain.
- Use fan-out and fan-in constructs with configurable all, any, quorum, or best-effort join policies.
- Apply semaphores and resource locks at the orchestration layer and honor downstream quotas.
- Cancel unnecessary in-flight work when a branch decision or sufficient evidence is reached.
- Preserve deterministic aggregation order where result ordering affects prompts or outputs.

Bottlenecks, risks, and safeguards

- Hidden dependencies can yield inconsistent snapshots. Use versioned reads or capture a common as-of time.

Parallel retries can create a thundering herd. Coordinate retry budgets and jitter.

Speculative calls can increase cost and expose unnecessary data. Base them on expected value and policy.

One slow branch can dominate tail latency. Use deadlines and partial-result rules.

Validation and operating metrics

Critical-path latency, achieved concurrency, and fan-out overhead.

Rate-limit errors, lock contention, and inconsistent-read incidents.

Cancelled speculative work and cost saved or wasted.

Join timeout rate and quality impact of partial results.

What the interviewer is testing: Whether you can use concurrency safely by modeling data dependencies, shared resources, and tail latency.

28. One tool returns a large amount of unstructured data that exceeds the context window. How should the agent process and summarize the response safely?

The task is to avoid lossy one-shot summarization and preserve provenance for later verification.

Model answer

The raw tool output should be stored as an immutable artifact outside the model context. A parser first extracts structure, metadata, sections, records, and errors. The system then creates searchable chunks or typed intermediate records and retrieves only the portions relevant to the active question. If a global summary is required, it uses hierarchical map-reduce or domain-specific aggregation, with every summary statement linked to source spans.

The agent should distinguish data reduction from interpretation. Deterministic code handles filtering, counting, deduplication, and numerical aggregation. Models summarize language or classify records in bounded batches. A verifier samples or checks critical claims against the source. Later steps can retrieve the original evidence rather than relying solely on a recursively compressed summary.

Architecture and implementation

Persist raw output with checksum, tool version, timestamp, and access controls.

Parse into structured records where possible and index by entities, time, fields, and source offsets.

Use query-focused retrieval before summarization and allocate context by relevance and coverage.

Create layered summaries: batch summaries, section summaries, and a final synthesis with citations.

Carry truncation, parse errors, and coverage metadata into the agent state.

Bottlenecks, risks, and safeguards

One-shot truncation can remove rare but critical evidence.

Recursive summaries can amplify an early mistake. Preserve source links and allow drill-down.

Untrusted tool output may contain prompt injection. Treat it as data and strip or isolate instructions.

Parallel batch summaries may use inconsistent terminology. Normalize entities and schema before synthesis.

Validation and operating metrics

Source coverage, claim support, and retrieval recall for known critical facts.

Compression ratio, context tokens, and summary latency.

Parse-error and truncation rates.

Disagreement between generated summaries and deterministic aggregates or sampled source checks.

What the interviewer is testing: Whether you can build an evidence-preserving data pipeline instead

of feeding an oversized blob to the model.

29. How would you prevent an agent from claiming that an external action succeeded when the corresponding tool call failed or never occurred?

This tests grounding final responses in execution receipts rather than model narrative.

Model answer

The model should not be the source of truth for execution status. The orchestrator owns a typed action ledger with states such as proposed, authorized, submitted, acknowledged, confirmed, failed, and ambiguous. Only the tool gateway can advance action states, and it must attach a signed or verifiable receipt, provider identifier, or post-condition result. The response generator receives this ledger and is instructed to describe only confirmed outcomes.

I would also generate user-facing status deterministically for critical actions. For example, a payment confirmation is templated from the transaction record, not written from memory. If a call failed or was never submitted, the response explicitly says so and distinguishes it from partial progress. A final consistency check compares every action claim with a ledger entry.

Architecture and implementation

- Use immutable action IDs and append-only status events from the tool gateway.
- Separate proposal text from execution status and store both.
- Require receipts or post-condition evidence before using verbs such as sent, created, updated, or paid.
- Implement a claim-to-action validator on final structured output.
- Use domain templates for high-impact confirmations and include reference numbers when safe.

Bottlenecks, risks, and safeguards

- A downstream API may acknowledge submission but later reject processing. Distinguish accepted from completed.
- The model may infer success from absence of an error. Require positive evidence.
- Cached status can be stale. Use current ledger state and mark eventual outcomes clearly.
- A compromised tool could forge success. Verify provider callbacks or reconcile with authoritative state.

Validation and operating metrics

- False success-claim and false failure-claim rates.
- Percentage of action statements backed by valid receipts.
- Ambiguous status age and reconciliation completion.
- Customer disputes caused by mismatched communication and external state.

What the interviewer is testing: Whether you enforce truthful action reporting through state and evidence rather than prompting the model to be honest.

30. A third-party API changes its schema without notice. How would your agent platform detect the change and prevent widespread workflow failures?

The expected answer covers contract boundaries, adapters, canaries, monitoring, and controlled degradation.

Model answer

All third-party APIs should sit behind versioned tool adapters owned by the platform or domain team. The adapter validates responses against an expected schema and normalizes them into a stable internal result type. Contract tests run regularly against a sandbox or synthetic account. At runtime, schema validation, unknown-field monitoring, parse errors, and semantic post-condition failures detect drift quickly.

When incompatibility is detected, a circuit breaker disables or degrades the affected tool rather than allowing models to interpret arbitrary changed payloads. Workflows receive a typed unavailable or incompatible error and follow an approved fallback: alternate provider, read-only mode, manual queue, or transparent failure. A new adapter version is tested against recorded fixtures and canaried before rollout.

Architecture and implementation

- Keep provider payloads inside adapters and expose stable internal schemas to agent code.
- Run scheduled contract tests, synthetic transactions, and schema-diff alerts.
- Validate both syntax and required semantics, such as identifier stability and status meaning.
- Use per-tool circuit breakers and feature flags for rapid containment and rollback.
- Maintain version compatibility and migration documentation in the tool registry.

Bottlenecks, risks, and safeguards

- Permissive parsing can hide a semantic change. Alert on unknown enums, missing invariants, and distribution shifts.
- A provider may change behavior without changing schema. Use post-condition and synthetic outcome checks.
- Automatic fallback can violate region or data policy. Approve fallbacks per task and tenant.
- Retry storms can amplify an outage. Stop retries for contract failures and apply backpressure.

Validation and operating metrics

- Contract-test pass rate, schema-drift detection time, and mean time to containment.
- Tool error rate and affected workflow volume by adapter version.
- Fallback success and quality degradation.
- Change failure rate and rollback time for adapter releases.

What the interviewer is testing: Whether you isolate external volatility behind contracts and can contain a change before it becomes an agent-wide incident.

SECTION 04

Agent State, Memory, and Context Management

These questions test whether the candidate can separate durable task state from model context, design trustworthy memory, control cross-tenant access, and recover long-running work after failures.

Section focus: Memory should be typed, consented, versioned, attributable, and retrievable for a specific purpose. Summaries and embeddings are indexes or caches, never the only source of truth.

31. What is the difference between working memory, episodic memory, semantic memory, conversation history, and external state in an agent system?

The interviewer is testing conceptual clarity and whether these forms of state are given different retention and trust policies.

Model answer

Working memory is the bounded information assembled for the current model call: goal, active step, relevant evidence, and recent observations. Conversation history is the sequence of user and assistant messages, which may help resolve references but is not necessarily canonical task state. Episodic memory stores records of prior interactions or completed tasks with time and context. Semantic memory stores durable facts or preferences abstracted from episodes, such as a user preference or organization policy.

External state is the authoritative operational record outside the model: workflow status, approvals, tool receipts, database entities, artifacts, and event logs. It must be durable and transactionally managed. The model context may include selected views of any of these, but the context itself should not be treated as persistence. Different categories require different consent, retention, update, retrieval, and deletion rules.

Architecture and implementation

- Keep working memory ephemeral and reconstruct it from canonical sources for each step.
- Store conversation history separately from structured task state and allow selective inclusion.
- Represent episodic memory with actor, timestamp, task, outcome, provenance, and retention metadata.
- Represent semantic memory as typed claims with source episodes, confidence, validity interval, and user controls.
- Use external state for anything that drives execution, authorization, recovery, billing, or audit.

Bottlenecks, risks, and safeguards

- Treating chat history as state loses tool receipts and creates ambiguity after summarization.
- Promoting one episode into a permanent fact can create memory contamination.
- Embedding stores do not enforce business consistency. Keep authoritative values in transactional systems.
- Mixing categories makes privacy deletion and retention difficult.

Validation and operating metrics

- Context relevance and omission rates for working memory.
- Memory correction, expiration, and deletion completion rates.
- Stale or unsupported semantic-memory usage.
- Task recovery success from external state without relying on transcript replay.

What the interviewer is testing: Whether you distinguish context, history, learned facts, past episodes,

and authoritative execution state and assign each an appropriate lifecycle.

32. An agent must resume a task after a process crash. What information should be checkpointed?

The expected answer includes enough state to resume safely without redoing irreversible work.

Model answer

I would checkpoint the task specification, plan version, active step, completed and pending steps, budgets, approvals, assumptions, selected model and prompt versions, and references to artifacts and evidence. For every tool action I would store the operation ID, arguments hash, authorization decision, attempt count, current status, result or error, and any downstream receipt. I would also store timers, callbacks, locks or leases, and cancellation state for long-running work.

Checkpoints should occur at state boundaries, especially before and after side effects. The recovery worker acquires a lease, reconstructs the materialized state from the event log or latest snapshot, and reconciles any action left in submitted or ambiguous status before proceeding. Model hidden reasoning need not be stored; reproducibility comes from inputs, outputs, state transitions, and evidence.

Architecture and implementation

- Use an append-only event log for state transitions plus periodic snapshots for fast recovery.
- Persist references to large artifacts in object storage rather than embedding them in the checkpoint.
- Record idempotency keys and post-condition status for every side-effecting action.
- Version workflow definitions and state schemas so old runs can replay or migrate safely.
- Use leases and heartbeats to prevent two workers from resuming the same task concurrently.

Bottlenecks, risks, and safeguards

- Checkpointing only after a tool call leaves an uncertainty window. Persist intent before submission and receipt after response.
- In-memory locks disappear on crash. Use durable coordination or optimistic concurrency.
- Workflow code changes can make old state incompatible. Use versioned handlers and migration tests.
- Storing secrets or raw sensitive payloads in checkpoints increases exposure. Store references and protected tokens.

Validation and operating metrics

- Recovery success, mean time to resume, and duplicate-side-effect rate.
- Checkpoint age at failure and event-replay duration.
- Lease conflict and split-brain incidents.
- Percentage of in-flight actions reconciled automatically after restart.

What the interviewer is testing: Whether you can make agent execution durable and reason about crash consistency around external side effects.

33. How would you design long-term memory for a personal assistant while allowing users to inspect, correct, delete, and expire stored memories?

This tests product consent, privacy, data modeling, and retrieval design together.

Model answer

I would make memory opt-in and purpose-specific. Candidate memories are extracted from interactions, but only allowed categories are stored, such as explicit preferences, recurring constraints, or user-approved facts. Each memory is a typed record with a human-readable statement, source episode, timestamp, confidence, sensitivity, scope, expiration policy, and status. Sensitive categories may be prohibited or require explicit confirmation.

The product should expose a memory center where users can see why a memory exists, edit it, delete it, pause memory, and clear categories or all history. Deletion must propagate to the primary store, vector index, caches, and downstream derived records according to documented retention limits. At retrieval time, the system uses purpose, recency, scope, and confidence and tells the user when a memory materially influenced an action.

Architecture and implementation

- Separate candidate extraction from a policy and consent gate before persistence.
- Use structured memory records with source links and version history rather than opaque text snippets alone.
- Maintain user, household, organization, task, and device scopes explicitly.
- Implement expiration, review reminders, and conflict resolution between newer and older memories.
- Create deletion workflows with tombstones and index invalidation to prevent reappearance.

Bottlenecks, risks, and safeguards

- Implicitly inferred sensitive traits can surprise or harm users. Restrict inference and require confirmation.
- Deleting a memory record but retaining its embedding or cache defeats user control.
- A preference valid in one context may be applied globally. Use scope and purpose constraints.
- Memory can become a covert instruction channel. Treat stored content as data and apply current policy at use time.

Validation and operating metrics

- Memory acceptance, correction, deletion, and expiration rates.
- Helpful-use versus irrelevant-use rate and user-reported surprise.
- Deletion propagation time and residual-memory audit results.
- Task-quality lift attributable to memory, segmented by category and scope.

What the interviewer is testing: Whether you can build useful personalization while preserving agency, transparency, scope, and end-to-end deletion.

34. An agent stores an incorrect user preference and keeps using it in future sessions. How would you prevent memory contamination?

The key is treating memory as revisable claims with provenance, not unquestioned facts.

Model answer

I would avoid writing a permanent preference from a single ambiguous statement. The memory extractor produces a candidate with confidence and source. Explicit statements such as "Always use metric units"

can be stored with high confidence; inferred preferences require confirmation or repeated evidence. At use time, the memory is presented as a suggestion with freshness and scope, not as an immutable instruction.

When the user contradicts or corrects a memory, the system creates a new version, invalidates the old one, and updates the retrieval index. It should not merely append both and leave the model to choose. A conflict resolver prioritizes explicit, recent, and context-matching evidence. The interface should let the user correct the preference directly, and future responses should acknowledge the corrected state when relevant.

Architecture and implementation

- Assign confidence, source type, scope, and expiration to every memory claim.
- Use confirmation thresholds for inferred or high-impact preferences.
- Maintain a single current record with version history and conflict links.
- Retrieve memories through policy that favors explicit, recent, and task-relevant claims.
- Record feedback when a memory is accepted, ignored, or corrected to improve extraction rules.

Bottlenecks, risks, and safeguards

- Appending contradictory memories creates nondeterministic behavior.
- Repeated model-generated statements can falsely appear as independent evidence. Trace all derived claims to original user sources.
- A wrong memory can affect consequential actions. Require current confirmation for high-impact use.
- Cached context may continue using the old value after correction. Invalidate session and retrieval caches.

Validation and operating metrics

- Incorrect-memory use, correction latency, and recurrence after correction.
- Precision of candidate memory extraction and confirmation acceptance.
- Conflict rate and resolution accuracy.
- User override and memory-related complaint rates.

What the interviewer is testing: Whether you model memory as uncertain, versioned, and user-correctable rather than a permanent transcript summary.

35. Should agent memory store raw conversations, generated summaries, structured facts, embeddings, or a combination?

A strong answer uses multiple representations with clear source-of-truth and privacy roles.

Model answer

I would use a combination, but each representation has a different purpose. Raw conversations provide audit and rich context but are expensive, sensitive, and difficult to retrieve precisely. Generated summaries reduce size but can omit or distort details. Structured facts support deterministic filtering, conflict resolution, and user editing. Embeddings provide semantic retrieval but are not human-readable or authoritative.

The canonical source should be the minimum raw or structured data needed for the product and policy. Structured memory records link to source spans. Summaries are derived views that can be regenerated. Embeddings index records and source material but can be deleted and rebuilt. Retention should be category-specific: some products may not retain raw conversations at all, while keeping only user-approved facts.

Architecture and implementation

- Store source episodes only when justified by product need and retention policy.
- Extract structured claims with provenance, confidence, validity, and scope.

- Generate task or period summaries from canonical records and label them as derived.
- Create embeddings over approved records, not unrestricted sensitive data by default.
- Encrypt, access-control, and delete every representation through one memory lifecycle service.

Bottlenecks, risks, and safeguards

- Keeping everything indefinitely creates privacy and discovery risk.
- Summary-only storage makes corrections and audits difficult.
- Embeddings can leak membership or semantic information and are easy to forget during deletion.
- Structured extraction can overstate uncertain language. Preserve qualifiers and source.

Validation and operating metrics

- Retrieval usefulness and precision by representation type.
- Storage cost, context-token savings, and regeneration time.
- Summary factuality and structured-extraction accuracy.
- Deletion completeness across raw, structured, summarized, embedded, and cached forms.

What the interviewer is testing: Whether you can assign canonical, derived, and index roles to memory representations and manage their different risks.

36. How would you attach provenance, confidence, timestamps, and expiration rules to agent memories?

The expected answer describes a memory schema and how metadata affects retrieval and use.

Model answer

I would model each memory as a claim rather than a blob. Required fields include subject, predicate, value, scope, source type and source pointer, extractor version, creation and observation time, confidence, sensitivity, validity interval, expiration policy, status, and superseded-by reference. Confidence should describe evidence quality, not just model probability. Explicit user statements, authoritative records, and model inferences receive different source weights.

Retrieval applies metadata before semantic similarity. It filters by user or tenant, purpose, sensitivity, scope, status, and validity. Ranking then combines relevance, recency, confidence, and source authority. Expired memories are omitted or routed for revalidation. When a memory influences a consequential action, the agent should surface its source or request confirmation.

Architecture and implementation

- Use immutable versions and a current-status pointer for updates and corrections.
- Track observed-at separately from stored-at so delayed ingestion does not imply freshness.
- Define expiration by category, such as short-lived location, medium-lived project preference, or durable explicit setting.
- Derive confidence from source authority, explicitness, consistency, and recency using calibrated rules or models.
- Log every retrieval and use with the memory version and decision context.

Bottlenecks, risks, and safeguards

- A numeric confidence without semantics creates false precision. Document how it is derived and calibrated.
- Timestamps can be ambiguous across observation, event, and ingestion time. Store all relevant times.
- Expiration without deletion may still expose data. Separate use validity from retention policy.
- Source links may contain sensitive content. Protect them with the same access controls as the memory.

Validation and operating metrics

- Confidence calibration and error rate by source type.
- Expired or superseded memory usage incidents.
- Provenance completeness and successful source resolution.
- Retrieval quality before and after metadata ranking.

What the interviewer is testing: Whether you can create a trustworthy memory data model and make provenance and validity operational in retrieval.

37. An agent has thousands of stored memories. How would you retrieve only those relevant to the current task?

This tests hybrid retrieval, purpose filtering, reranking, and context-budget management.

Model answer

I would first derive a memory query from the task specification, including entities, time range, domain, and requested action. Hard filters apply identity, tenant, consent, purpose, scope, sensitivity, validity, and current status. Candidate retrieval combines lexical matching for exact names and identifiers, vector search for semantic similarity, and relational lookups for structured attributes. A reranker then considers task relevance, source authority, recency, confidence, and diversity.

The context builder selects a small evidence set within a token budget and groups related claims. It avoids redundant memories and includes contradictions when they are material. High-impact decisions may require direct source retrieval rather than a memory summary. The system should also support no relevant memory as a valid outcome.

Architecture and implementation

- Generate query features from the current goal and active plan step, not only the latest user message.
- Use metadata filters before similarity search to enforce privacy and validity.
- Apply hybrid retrieval and a task-specific reranker.
- Deduplicate semantically equivalent memories and diversify across relevant aspects.
- Include provenance and confidence in the context representation and reserve tokens for current evidence.

Bottlenecks, risks, and safeguards

- Vector similarity may retrieve sensitive but irrelevant memories. Policy filters must be mandatory.
- Recency can suppress a durable explicit preference. Ranking should be category-aware.
- Over-personalization can override current instructions. Current user intent has priority.
- A large memory query can create latency. Use partitioning, precomputed metadata, and caching with careful invalidation.

Validation and operating metrics

- Recall and precision on labeled memory-relevance cases.
- Irrelevant-memory use, privacy-filter violations, and no-memory accuracy.
- Retrieval and reranking latency plus tokens added to context.
- Task-quality lift and user acceptance attributable to retrieved memory.

What the interviewer is testing: Whether you can retrieve memory as governed evidence rather than performing unconstrained semantic search over a personal archive.

38. How would you prevent memories belonging to one tenant or user from being retrieved for another?

This is a multi-tenant isolation question; prompt instructions are not an acceptable primary control.

Model answer

Isolation begins with identity propagated from the authenticated request, not a tenant ID supplied by the model. Every memory record and index entry carries tenant and subject identifiers. Retrieval services require those identifiers in the authorization context and apply them as mandatory filters inside the storage query. For high-assurance environments, I would use separate indexes, schemas, encryption keys, or clusters for stronger physical or cryptographic isolation.

The model never receives credentials that can broaden scope. Cache keys include tenant, user, purpose, memory version, and policy version. Background jobs and evaluation pipelines preserve the same boundaries. Defense in depth includes row-level security, service authorization, audit logs, synthetic cross-tenant tests, and alerts for impossible identity combinations.

Architecture and implementation

- Use a trusted identity service and pass signed claims through the orchestrator to memory services.
- Enforce tenant and subject filters at the database or index layer, not only in application prompts.
- Partition storage according to risk and scale, with per-tenant encryption for sensitive tiers.
- Namespace caches, queues, object paths, and observability data consistently.
- Run automated canary documents and adversarial tests that attempt cross-tenant retrieval.

Bottlenecks, risks, and safeguards

- Post-filtering retrieved results can expose data in logs or model context before filtering.
- Shared caches are a common leakage path. Include identity and authorization scope in every key.
- Asynchronous jobs can lose request context. Persist signed execution identity with the task.
- Embeddings copied into evaluation datasets may escape tenant controls. Apply governed export and de-identification.

Validation and operating metrics

- Cross-tenant access attempts and successful leakage rate, expected to be zero.
- Authorization-filter coverage and cache-isolation test results.
- Audit-log completeness for memory reads and writes.
- Time to revoke access and invalidate affected caches or indexes.

What the interviewer is testing: Whether you enforce isolation in storage, identity, caching, and asynchronous execution rather than relying on model behavior.

39. A conversation summary removes a critical constraint from the user's original instruction. How would you detect and prevent this type of memory drift?

The key is not using a free-form summary as the only representation of requirements.

Model answer

I would extract critical constraints into a structured task contract before summarization. Fields such as prohibited actions, budget, deadline, recipient, data scope, approval requirement, and output format are preserved independently and injected into every relevant model call. The narrative summary is a convenience for conversational continuity, not the authority for execution.

When summaries are generated, a verifier compares them with the source turns and the structured contract. It checks required-constraint coverage, entity consistency, and unresolved commitments. Summaries are versioned and can be regenerated from source history. Before a high-impact action, the orchestrator uses the canonical constraint record and may show the exact action for confirmation.

Architecture and implementation

- Classify and pin immutable or safety-critical constraints at task creation and update them only through explicit state transitions.
- Use a summary schema with sections for goals, constraints, decisions, open questions, and completed actions.
- Run coverage checks that ensure every active constraint is represented or referenced.
- Retain source message pointers for each summarized decision and constraint.
- Rebuild summaries periodically from source events rather than repeatedly summarizing summaries.

Bottlenecks, risks, and safeguards

- A model may reinterpret a constraint during extraction. Use confirmation for ambiguous high-impact fields.
- Pinned constraints can become stale when the user changes intent. Version them and record explicit supersession.
- Long histories may be unavailable due to retention policy. Preserve the minimum structured contract required for execution.
- Validation by the same prompt can repeat omissions. Add deterministic field coverage and independent spot checks.

Validation and operating metrics

- Constraint coverage, entity consistency, and summary factuality.
- Action blocks caused by missing or conflicting constraints.
- Drift rate across repeated summarization cycles.
- User corrections and incidents attributable to summary omission.

What the interviewer is testing: Whether you separate canonical constraints from conversational summaries and can verify compression against source state.

40. How would you decide which information belongs in the model context, a state database, a vector store, an event log, or a relational database?

This tests data architecture and the ability to assign each store a clear consistency and retrieval role.

Model answer

The model context should contain only the information required for the current decision and is ephemeral. A task-state database holds the current materialized workflow state and typed records needed for control. A relational database is appropriate for authoritative entities, constraints, transactions, and queries requiring consistency or joins. A vector store is an index for semantic candidate retrieval over unstructured or memory content, not the source of truth. An event log records immutable state transitions and external events for replay, audit, and rebuilding materialized state.

Object storage handles large documents, tool outputs, code, and generated artifacts, referenced by IDs from the other stores. I choose based on consistency, query pattern, update semantics, retention, and recovery needs. Data may appear in several representations, but ownership must be explicit and derived copies must have invalidation and deletion paths.

Architecture and implementation

- Model context: bounded view, reconstructed per call, with version references.

State store: current task, plan, step status, budgets, approvals, and active locks.

Relational source: users, accounts, policies, financial records, and other authoritative structured data.

Vector index: semantic retrieval candidates with metadata and pointers to canonical content.

Event log: append-only commands, results, transitions, and callbacks; object store: large immutable artifacts.

Bottlenecks, risks, and safeguards

Using a vector store as the only database makes updates, uniqueness, and access control difficult.

Duplicated derived data becomes stale without versioning and invalidation.

Putting complete records in prompts increases privacy exposure and token cost.

An event log without materialized views can make operational queries slow; a state table without an event log weakens replay.

Validation and operating metrics

Consistency and staleness incidents across stores.

Context size, retrieval latency, and database query performance.

Replay and state-rebuild success.

Deletion propagation and access-control coverage for all representations.

What the interviewer is testing: Whether you can map data to stores by consistency and access pattern and maintain a clear source-of-truth hierarchy.

SECTION 05

Agent Loops, Termination, and Recovery

These scenarios cover infinite loops, progress detection, execution budgets, failure classification, safe resumption, inconsistent observations, and compensation after partial side effects.

Section focus: Termination should be based on explicit success, blocked state, risk, and diminishing expected value. Recovery begins by reconciling external state before any retry.

41. An agent repeatedly calls a search tool because it believes it needs more information. How would you detect and stop the loop?

The interviewer expects semantic loop detection, information-gain measurement, and a useful exit path.

Model answer

I would track each search query, normalized intent, retrieved source set, and the claims or unresolved questions it addresses. A progress monitor detects exact repetition, semantically similar queries, repeated source domains, and low marginal information gain. The search step must update a structured evidence table or unresolved-question list; if it adds no new supported claim or reduces no uncertainty, it does not count as progress.

The agent receives a search budget and stop criteria such as minimum source coverage, sufficient confidence for the requested decision, or an explicit inability to find evidence. After repeated low-gain searches, the system changes retrieval strategy, broadens or narrows the query once, asks the user for missing context, or returns an evidence-limited answer. It should never conceal that the evidence is incomplete.

Architecture and implementation

- Fingerprint queries and retrieved documents and compute overlap across iterations.
- Maintain a claim-evidence matrix and unresolved-question register.
- Estimate information gain from new sources, contradiction resolution, and coverage of required facts.
- Use separate budgets for query rewriting, web search, page retrieval, and document analysis.
- Define terminal outcomes for sufficient evidence, no evidence, conflicting evidence, blocked access, and budget exhaustion.

Bottlenecks, risks, and safeguards

- Exact-match detection misses paraphrased loops. Add semantic and result-set similarity.
- A new URL may duplicate the same underlying report. Deduplicate by content and provenance.
- Stopping too early can reduce recall. Use task-specific evidence thresholds and risk-aware budgets.
- Search results may contain injected instructions. Treat content as untrusted evidence and isolate it from control instructions.

Validation and operating metrics

- Search calls per task, low-information search rate, and loop interventions.
- Marginal evidence gain and required-claim coverage by iteration.
- Task quality before and after stop-policy changes.
- Percentage of tasks ending with transparent insufficiency rather than unsupported answers.

What the interviewer is testing: Whether you can define progress for research and stop based on evidence gain instead of only a hard iteration cap.

42. What termination conditions would you implement beyond a maximum number of tool calls?

A strong answer includes success, impossibility, blocked state, safety, economics, and user-driven termination.

Model answer

I would define positive completion as verified satisfaction of task-level success criteria and all mandatory invariants. Other valid terminal states include awaiting user input, awaiting approval, blocked by permission, unavailable dependency, insufficient evidence, conflicting evidence, policy refusal, cancelled, deadline exceeded, budget exhausted, and unrecoverable failure. Each state has a user-facing explanation and a machine-readable reason.

I would also stop on diminishing expected value: repeated no-progress transitions, exhausted strategies, or an estimated probability of success that no longer justifies cost or risk. Safety termination can override any other goal when the requested or emergent action crosses policy. Time, token, monetary, and external-service budgets are independent because a run may be cheap in tokens but expensive in API calls or human review.

Architecture and implementation

- Define success criteria and terminal-state taxonomy per workflow.
- Check completion against validated state and receipts, not model self-report.
- Add no-progress, repeated-action, deadline, and expected-value conditions.
- Support pause states that preserve work while waiting for signals or approvals.
- Return completed artifacts, unresolved items, and safe next steps for every non-success terminal state.

Bottlenecks, risks, and safeguards

- A single global call limit ignores task complexity and risk.
- Stopping on low confidence alone may be poorly calibrated. Combine confidence with evidence and failure history.
- An agent can mark a task complete after generating output but before external state is confirmed.
- A pause without expiration or ownership can become a resource leak. Add deadlines and cleanup policy.

Validation and operating metrics

- Distribution and correctness of terminal states.
- False completion, premature termination, and runaway execution rates.
- Resource consumption at termination and value of partial artifacts.
- Resume success for paused tasks and cleanup of abandoned tasks.

What the interviewer is testing: Whether you understand termination as a domain state model rather than a simple loop counter.

43. How would you measure whether an agent is making meaningful progress toward its goal?

The challenge is defining domain-grounded progress rather than counting generated text or tool calls.

Model answer

I would derive progress variables from the task definition. Examples include verified requirements completed, dependencies resolved, evidence gaps closed, tests passing, records processed, approved milestones reached, or distance from a target state. These variables should be updated by validated artifacts and tool results. A tool call is activity, not progress, unless it changes one of these measures.

For less structured tasks, I would maintain a rubric-based state: goal coverage, evidence coverage, contradiction count, unresolved questions, and output readiness. The system compares current values with a prior checkpoint and estimates marginal progress per unit of cost. A progress score is useful for control, but the underlying dimensions remain visible so the agent cannot game one aggregate number.

Architecture and implementation

- Define milestone and invariant fields in the task schema before execution.
- Update progress only after validation or authoritative tool confirmation.
- Track both monotonic measures and regressions caused by invalidated assumptions.
- Calculate progress velocity and cost per resolved unit over a sliding window.
- Use domain-specific evaluators for code tests, research claims, data rows, or workflow states.

Bottlenecks, risks, and safeguards

- Generic completion percentages can be fabricated and misleading.
- Some exploration initially increases unresolved questions. Permit bounded negative progress when it reveals important risk.
- A single metric can be gamed, such as maximizing document count instead of evidence quality.
- Validation delays can make real progress appear absent. Represent pending and confirmed states separately.

Validation and operating metrics

- Milestone completion and progress velocity.
- No-progress windows, regression frequency, and validation backlog.
- Cost and latency per verified unit of progress.
- Correlation between progress score and eventual task success.

What the interviewer is testing: Whether you can ground progress in observable state changes and use it for control and economics.

44. An agent alternates between two tools without improving its state. How would you identify this oscillation?

This tests state-transition analysis and strategy change after repeated equivalent states.

Model answer

I would store a compact signature for each iteration containing active goal, unresolved requirements, selected tool, normalized arguments, result category, and material state changes. Oscillation is detected when the system revisits a recent state signature or cycles through a short sequence with no improvement in validated progress. Semantic similarity is useful because arguments may vary slightly while the strategy remains equivalent.

When detected, the orchestrator blocks immediate repetition and invokes a recovery policy. It can compare the two tools outputs, identify the unresolved assumption, switch to a different capability, ask for missing input, or terminate. The recovery prompt includes the cycle evidence and explicitly prohibits the failed transition pattern. If the tools are side-effecting, the system first reconciles their external state before any alternate action.

Architecture and implementation

- Maintain a bounded history of state and action fingerprints.
- Detect exact cycles, semantic cycles, and repeated result categories.
- Require a new evidence item or changed strategy before allowing a blocked action again.
- Assign cycle-specific recovery actions and escalation thresholds.
- Include oscillation events in traces and offline failure clustering.

Bottlenecks, risks, and safeguards

- Normal polling can look like a loop. Label expected wait or poll states with interval and deadline.
- Slightly changing parameters can bypass exact fingerprints. Normalize and compare intent and outcome.
- Forcing a new tool can be worse if the root cause is missing permission or data. Diagnose the blocker first.
- Two tools may be implementing a legitimate reconciliation process. Model known workflows explicitly.

Validation and operating metrics

- Cycle detection rate, false-positive rate, and average cycle length.
- Cost and latency consumed before detection.
- Recovery success after cycle intervention.
- Most common state and tool pairs involved in oscillation.

What the interviewer is testing: Whether you can detect semantic cycles from state transitions and choose a reasoned recovery rather than simply increasing limits.

45. How would you design token, time, financial, and tool-call budgets for agent execution?

The expected answer treats budget as a multi-dimensional policy with reservation, tracking, and graceful exhaustion.

Model answer

I would create a budget envelope when the task is admitted, based on task class, user tier, predicted complexity, risk, and expected business value. It contains separate limits for model tokens and cost, elapsed and compute time, total and per-tool calls, external API spend, data volume, and human-review usage. The planner estimates consumption by step and reserves budget for mandatory validation and final reporting.

The orchestrator meters actual use in real time. Each step has a local allowance and cannot spend the entire task budget. When remaining budget falls below a threshold, the agent must choose among a cheaper model, reduced retrieval, partial result, approval for more budget, or termination. High-risk checks cannot be skipped merely to stay within cost; instead the action should not execute.

Architecture and implementation

- Define hard safety and financial limits plus softer optimization targets.
- Reserve budget for validation, compensation, and user-facing completion before exploratory work.
- Apply per-tenant quotas, per-tool rate limits, and concurrency budgets in addition to per-task limits.

Expose remaining budget and predicted step cost to the planner in a structured form.

Record budget overrides with identity, reason, expiration, and affected task version.

Bottlenecks, risks, and safeguards

One global dollar limit misses latency or external service constraints.

A planner can underestimate expensive fan-out. Use historical distributions and safety margins.

Hard cutoff during a side effect can leave inconsistent state. Finish or reconcile the current transaction safely.

Budget pressure can bias the agent toward unsupported answers. Prefer transparent partial completion.

Validation and operating metrics

Budget prediction error and percentage of runs within each limit.

Cost per successful task, marginal quality per additional spend, and budget-exhaustion outcomes.

Resource use by plan step, model, tool, tenant, and task class.

Safety or validation steps skipped, which should be zero for required controls.

What the interviewer is testing: Whether you can manage agent economics and capacity without allowing budget optimization to bypass correctness or safety.

46. An agent reaches its execution limit without completing the task. What information should it return to the user?

This tests transparent partial completion and preservation of reusable work.

Model answer

The response should state that the task is incomplete and identify the limiting condition, such as time, budget, unavailable service, or unresolved evidence. It should summarize the original goal, verified work completed, artifacts produced, actions actually executed, unresolved items, and any external state that requires attention. It must not present a partial conclusion as final.

The system should offer safe continuation options derived from state: resume with a larger budget, provide missing information, approve a specific action, wait for a dependency, or hand the case to a person. It includes a task or checkpoint reference so continuation uses the same durable state rather than starting over. Sensitive internal traces and raw chain-of-thought are not needed; concise evidence and decision records provide auditability.

Architecture and implementation

Generate the status report from canonical step and action state, not a model recollection.

Separate confirmed results, tentative findings, and unverified hypotheses.

Link deliverable artifacts and cite relevant evidence where appropriate.

List side effects with exact status, including ambiguous or pending operations.

Provide a resume token or task ID and the conditions required to continue.

Bottlenecks, risks, and safeguards

The model may hide failure with confident prose. Use deterministic status fields and templates.

Exposing internal security details can increase risk. Report user-relevant causes without leaking secrets or defenses.

A resume may repeat a side effect if the checkpoint lacks operation IDs.

A partial result can be harmful if used as complete. Label scope and uncertainty prominently.

Validation and operating metrics

- User comprehension and successful resume rate after incomplete runs.
- Accuracy of reported completed and pending actions.
- Reuse of partial artifacts versus duplicated work.
- Complaints or downstream errors caused by misunderstood partial results.

What the interviewer is testing: Whether you can fail usefully and truthfully while preserving state and avoiding duplicated execution.

47. An agent completes eight of ten steps and then crashes. How would it resume without repeating side-effecting operations?

The answer should describe durable step state, idempotency, action reconciliation, and leases.

Model answer

Every step transition and tool action should already be durably recorded. On restart, a worker acquires the task lease, loads the latest snapshot and subsequent events, and identifies states that were complete, running, or ambiguous. Completed steps with validated artifacts are not rerun. For each side-effecting action, the worker uses the stable operation ID to query downstream status or replay idempotently before deciding whether the step is complete, failed, or still pending.

Only after reconciliation does the orchestrator schedule remaining ready steps. If the workflow definition changed, the run uses its original version or an explicit migration. The agent context is rebuilt from canonical state and artifacts. A crash-recovery test suite should simulate failure before submission, after submission but before receipt, after receipt but before checkpoint, and during validation.

Architecture and implementation

- Persist intent before side-effect submission and receipt or status after it.
- Assign stable idempotency keys and request hashes to all writes.
- Use step-level checkpoints, durable locks, and lease expiry for worker takeover.
- Mark in-flight actions as ambiguous until reconciled with authoritative systems.
- Version workflows and maintain replay-safe code for active runs.

Bottlenecks, risks, and safeguards

- Marking a step complete before validating the side effect can hide failure.
- A stale lease can allow concurrent workers. Use fencing tokens or optimistic version checks.
- Retrying under a new operation ID creates duplicates.
- Rebuilding context from a partial transcript can omit approvals and constraints.

Validation and operating metrics

- Crash-recovery success and duplicate-action rate.
- Time to reconcile in-flight actions and resume useful work.
- Lease conflict, replay, and migration failures.
- Coverage of injected-failure test points.

What the interviewer is testing: Whether you can provide effectively-once business behavior over at-least-once execution and process crashes.

48. How would you distinguish retryable failures, recoverable failures, user-correctable failures, and terminal failures?

This tests a practical error taxonomy and mapping from errors to control behavior.

Model answer

Retryable failures are transient conditions where repeating the same operation is safe and likely to succeed, such as a rate limit or network reset, subject to idempotency and backoff. Recoverable failures require a strategy change, alternate tool, refreshed credential, replanning, or compensation. User-correctable failures require information, authorization, or a changed request from the user. Terminal failures mean the goal is impossible, prohibited, invalid, or no safe recovery path remains.

The tool adapter should return typed errors with source, retryability, ambiguity, recommended delay, and whether a side effect may have occurred. The orchestrator combines that with operation risk and workflow state. The model may explain or propose alternatives, but it does not reinterpret a definite terminal or policy error into a retry.

Architecture and implementation

- Define a cross-platform error envelope and map provider-specific errors at adapters.
- Separate transport, authentication, authorization, validation, business, quota, dependency, and policy failures.
- Attach retry budget, backoff, idempotency requirement, and escalation policy to error classes.
- Persist the failure sequence so repeated recovery attempts can be limited.
- Return actionable, non-sensitive messages to users for correctable and terminal states.

Bottlenecks, risks, and safeguards

- HTTP status alone may not describe business success or ambiguity.
- Retrying authorization or validation errors wastes capacity and may trigger abuse controls.
- A recoverable error can become terminal after deadline or budget exhaustion.
- Provider error messages may contain sensitive data. Normalize and redact before logging or prompting.

Validation and operating metrics

- Error-classification precision and recovery success by class.
- Unproductive retries, retry amplification, and time to terminal decision.
- User-correction completion and abandonment rate.
- Incidents caused by misclassified ambiguous or non-retryable errors.

What the interviewer is testing: Whether you can turn heterogeneous failures into deterministic recovery behavior and safe user communication.

49. An agent receives inconsistent results from the same external system. Should it retry, compare results, escalate, or continue?

The candidate should diagnose consistency semantics and decision materiality before choosing a response.

Model answer

I would first determine whether variation is expected: eventual consistency, changing time-series data, nondeterministic search, replica lag, or a truly conflicting source of truth. Results should include timestamps, versions, request IDs, and provenance. If the operation is read-only and the discrepancy is

likely transient, a bounded retry against a consistent-read or authoritative endpoint may resolve it. Repeating the same request without changing consistency conditions is not useful.

The decision depends on materiality. For low-impact summarization, the agent can note the latest timestamp and continue. For a payment, entitlement, compliance status, or other high-impact decision, it should pause until the authoritative state is reconciled or a human resolves the conflict. The agent should preserve both observations and never average or choose one merely because it is convenient.

Architecture and implementation

- Capture version, as-of time, endpoint, region, and request ID with every observation.
- Define authoritative endpoints and acceptable staleness per data domain.
- Use quorum or cross-check logic only where the domain supports it.
- Route material conflicts to a reconciliation workflow rather than normal agent reasoning.
- Invalidate dependent conclusions when the selected source value changes.

Bottlenecks, risks, and safeguards

- Blind retries can amplify load during an incident and still return different replicas.
- Treating the newest value as correct may be wrong if timestamps reflect ingestion rather than event time.
- An LLM can rationalize conflicting results. Require explicit source authority rules.
- Continuing with an unresolved conflict can make later side effects unsafe.

Validation and operating metrics

- Conflict frequency, resolution time, and authoritative-source agreement.
- Retry effectiveness and load caused by consistency retries.
- Tasks paused or reversed because of stale or inconsistent data.
- Dependent-artifact invalidation completeness.

What the interviewer is testing: Whether you understand data consistency and can make risk-aware decisions rather than treating repeated API output as ground truth.

50. How would you design compensation logic for a workflow in which some irreversible actions have already occurred before a later step fails?

The expected answer uses saga-style thinking and acknowledges that compensation is not always a true rollback.

Model answer

I would model the workflow as a saga. Each side-effecting step declares whether it is reversible, its compensation action, the compensation deadline, required authority, and residual effects. Before execution, the planner orders operations to minimize irreversible exposure and places commit points after validation and approval. If a later step fails, the orchestrator executes compensations in reverse dependency order where safe.

Compensation is a new business action, not a database rollback. Refunding a payment, sending a correction, revoking access, or opening a case can have fees, delays, and legal consequences. Therefore the system validates and logs compensation just like the original action. If no automatic compensation exists, the workflow enters a contained incident state, stops further actions, notifies an owner, and presents the exact external state.

Architecture and implementation

- Record compensation metadata and tests in the tool registry for every write operation.

- Separate prepare, commit, and compensate nodes and bind them to the same business operation.
- Use reverse-topological compensation with dependency and authorization checks.
- Preserve original and compensating receipts for audit and customer communication.
- Provide manual remediation playbooks for non-compensable or partially compensable actions.

Bottlenecks, risks, and safeguards

- A compensation can fail or create another side effect. Give it independent retry and escalation policy.
- Not all actions are reversible, and timing may matter. Surface residual risk before the initial commit.
- Automatic compensation can be worse than leaving state unchanged. Use domain policy and materiality thresholds.
- Concurrent external changes can make the compensation stale. Validate current state before applying it.

Validation and operating metrics

- Compensation success, time to consistent state, and residual-impact rate.
- Workflows entering manual remediation and age of unresolved incidents.
- Original actions reordered or prevented by pre-commit validation.
- Customer, financial, and compliance impact from partial-failure cases.

What the interviewer is testing: Whether you can manage partial failure with sagas, explicit residual risk, and auditable remediation rather than promising impossible rollback.

SECTION 06

Multi-Agent Systems

These questions examine when specialization and concurrency justify multiple agents, how they coordinate, resolve conflict, share evidence, and remain isolated when one component is wrong or compromised.

Section focus: Use multiple agents only when decomposition, parallelism, specialization, or independent review produces measured value. Prefer structured artifacts and controlled orchestration over free-form agent conversations.

51. When does a multi-agent system provide a genuine advantage over a single agent with multiple tools?

The interviewer is testing whether the candidate can justify multi-agent overhead with measurable benefits.

Model answer

A multi-agent design is useful when subtasks require materially different capabilities, contexts, permissions, or execution lifecycles; when independent work can run in parallel; or when adversarial review benefits from separation. Examples include a researcher with broad read access, a code executor in a sandbox, and a release approver with no code-writing authority. It may also help when each specialist can operate with a smaller, cleaner context than one general agent.

It is not justified merely because roles sound natural. A single agent with tools is usually simpler, cheaper, and easier to evaluate. I would establish a single-agent baseline and add specialists only where traces reveal context interference, tool-selection confusion, bottlenecked parallelism, or a need for independent control. The coordinator should exchange typed tasks and artifacts, not encourage unconstrained debate.

Architecture and implementation

- Identify separable work with distinct inputs, outputs, permissions, and validation criteria.
- Use a central task graph or workflow to allocate and join work rather than relying on agents to negotiate all control decisions.
- Give specialists least-privilege tool sets and bounded context.
- Use independent reviewer roles only when their error correlation is lower than the producer role.
- Benchmark quality, latency, cost, and failure recovery against a single-agent and deterministic baseline.

Bottlenecks, risks, and safeguards

- Agent-to-agent messages consume context and can amplify hallucinations.
- Role labels do not guarantee real specialization if all agents use the same model and evidence.
- More agents increase scheduling, observability, and security surface.
- A coordinator can become a serial bottleneck or single point of failure.

Validation and operating metrics

- Incremental task-success lift and error reduction over a single-agent baseline.
- Parallel speedup, communication tokens, and orchestration overhead.
- Duplicate work, handoff failures, and conflict-resolution frequency.
- Security boundary violations and blast radius of specialist failures.

What the interviewer is testing: Whether you can distinguish real specialization and independence from unnecessary role-play and complexity.

52. Design a multi-agent system containing researcher, analyst, writer, and reviewer agents. How would they communicate and share evidence?

The expected answer is an artifact-driven pipeline with provenance and controlled iteration.

Model answer

I would use a coordinator backed by a task graph. The researcher receives explicit questions and produces an evidence package: source identifiers, excerpts, timestamps, credibility metadata, and unresolved conflicts. The analyst consumes that package and produces structured claims, calculations, assumptions, and confidence. The writer turns approved claims into a narrative but cannot introduce unsupported factual claims. The reviewer checks the draft against the claim-evidence graph and rubric and returns findings rather than directly rewriting everything.

Communication occurs through versioned artifacts in shared storage and event notifications, not one growing chat. Every claim has provenance and an owner. The coordinator controls iteration: critical unsupported claims return to research; analytical errors return to analysis; style issues return to writing. Budgets and maximum review cycles prevent endless handoffs.

Architecture and implementation

- Define schemas for research brief, source record, claim, calculation, draft section, and review finding.
- Store artifacts in a shared evidence repository with immutable versions and access controls.
- Pass compact references and task-specific views rather than full histories between agents.
- Create routing rules from review finding type and severity to the responsible agent.
- Separate factual approval from stylistic approval and keep a final deterministic citation check.

Bottlenecks, risks, and safeguards

- The writer may add plausible transitions that imply unsupported facts. Validate claims at sentence or assertion level.
- Research quantity can substitute for source quality. Rank authority and diversity, not document count.
- The reviewer may share the producer bias. Use different evidence views, models, or deterministic checks for important dimensions.
- Shared artifacts can leak restricted sources. Apply per-source permissions through every derived artifact.

Validation and operating metrics

- Claim support, source quality, calculation accuracy, and review defect precision.
- Handoff latency, artifact reuse, and number of revision cycles.
- Unsupported additions introduced by the writer.
- End-to-end quality and cost compared with one agent performing all roles.

What the interviewer is testing: Whether you can turn roles into enforceable contracts and an evidence lineage instead of a free-form group chat.

53. How would you prevent several agents from duplicating the same work?

This tests centralized work ownership, deduplication, and shared artifact discovery.

Model answer

The coordinator should maintain a canonical work graph with unique task IDs, owners, status, expected output, and dependency. Agents claim tasks through leases or assignments rather than independently deciding what to do from the same broad goal. Before creating a new subtask, the planner searches existing tasks and artifacts for equivalent scope. Work signatures can include entities, time range, requested operation, and output schema.

Shared results are published to an artifact catalog with provenance and validity. Agents retrieve and reuse them instead of repeating expensive searches or calculations. For intentional redundancy, such as independent verification, the task is labeled as such and isolated so one result does not bias the other. The join step compares outputs and resolves differences.

Architecture and implementation

- Use a central task registry with unique constraints and assignment leases.
- Normalize task specifications and compute structural or semantic duplicate signatures.
- Expose an artifact catalog indexed by task, entities, versions, and evidence coverage.
- Distinguish reuse, cooperative partitioning, and deliberately independent replication.
- Cancel or merge duplicate tasks when detected and preserve the best existing progress.

Bottlenecks, risks, and safeguards

- Semantic deduplication may merge tasks with different scope or authority. Compare structured constraints before merging.
- Agents can begin work before registry writes become visible. Use transactional claim operations.
- Reusing stale artifacts can be worse than recomputing. Track validity and source versions.
- Over-deduplication removes useful independent review. Mark intentional replication explicitly.

Validation and operating metrics

- Duplicate-task rate, compute and tool cost wasted, and cancellation effectiveness.
- Artifact reuse and stale-reuse incidents.
- Task-claim conflicts and lease expirations.
- Quality delta from intentional independent replication.

What the interviewer is testing: Whether you manage multi-agent work like a distributed scheduler with ownership and reusable artifacts.

54. In a multi-agent system, two agents produce conflicting conclusions. How should the orchestrator resolve the disagreement?

The answer should resolve evidence and assumptions, not rely on majority vote or model confidence alone.

Model answer

The orchestrator should decompose the disagreement into claims, sources, assumptions, calculations, and time or scope. It first checks whether the agents answered the same question with the same inputs. Many conflicts come from different definitions, document versions, or effective dates. Deterministic calculations

are recomputed from source data. Factual claims are compared by source authority and support. Judgment differences are presented with assumptions and decision criteria.

If one conclusion is clearly unsupported, it is rejected. If evidence remains genuinely ambiguous, the system can request targeted research, use an independent adjudicator with a narrow rubric, ask a domain expert, or present multiple supported outcomes. The coordinator should never average incompatible answers or choose the most confident tone.

Architecture and implementation

- Require agents to output structured claims, evidence references, assumptions, and uncertainty.
- Normalize definitions, time ranges, and data versions before comparison.
- Use deterministic recomputation for numerical or rule-based disputes.
- Route unresolved conflicts based on materiality to research, adjudication, human review, or transparent multi-view output.
- Record the resolution rule and rejected evidence for audit and evaluator improvement.

Bottlenecks, risks, and safeguards

- Majority voting fails when agents share the same model or source bias.
- An LLM judge may favor style or verbosity. Give it a claim-level rubric and source evidence.
- Forcing consensus can hide legitimate uncertainty.
- Agents may cite the same underlying source through different URLs. Deduplicate provenance.

Validation and operating metrics

- Conflict rate, root-cause distribution, and resolution accuracy.
- Human agreement with automated adjudication and reversal rate.
- Time and cost to resolve material disagreements.
- Frequency of appropriately preserved uncertainty.

What the interviewer is testing: Whether you resolve conflicts through evidence, definitions, and authority rather than voting among fluent outputs.

55. Would you use a centralized coordinator, peer-to-peer communication, a shared blackboard, or an event-driven architecture for multi-agent collaboration?

This tests topology selection based on control, scale, latency, and fault tolerance.

Model answer

A centralized coordinator is the best default for bounded business workflows because it provides one plan, consistent policy, clear ownership, and simple audit. A shared blackboard works well when specialists contribute artifacts asynchronously and discover work from shared state. Event-driven architecture is appropriate for long-running, high-scale systems with external callbacks and independently deployable workers. Peer-to-peer communication is useful only for constrained collaborative protocols because it complicates authorization, termination, and global consistency.

I often combine a durable coordinator with an event bus and shared artifact store. The coordinator owns the task graph and policy; events wake specialists; the blackboard stores versioned results. Agents do not directly grant authority to one another. Peer messages, if allowed, are typed requests routed through the platform so they remain observable and rate limited.

Architecture and implementation

- Choose a single owner for task state and termination even when execution is decentralized.

- Use events for asynchronous readiness and callbacks, with deduplication and ordering keys.
- Use a blackboard for immutable or versioned artifacts and claims, not uncontrolled mutable notes.
- Restrict peer communication to schemas and capability requests enforced by the coordinator.
- Partition coordinators and event streams by tenant or workflow while preserving global policy services.

Bottlenecks, risks, and safeguards

- A central coordinator can bottleneck or fail. Make it stateless where possible over durable state and scale by partition.
- Event systems introduce duplicates and reordering. Consumers must be idempotent.
- A shared blackboard can become an uncurated context dump. Add schemas, ownership, and retention.
- Peer-to-peer systems can develop loops and unauthorized delegation.

Validation and operating metrics

- Coordinator throughput, queue delay, event redelivery, and task-state consistency.
- Communication volume and artifact reuse by topology.
- Recovery time after worker or coordinator failure.
- Unauthorized or cyclic inter-agent requests.

What the interviewer is testing: Whether you can select and combine distributed coordination patterns with clear state ownership.

56. How would you prevent the context shared between agents from growing without limit?

The expected answer uses artifact references and task-specific context assembly instead of forwarding transcripts.

Model answer

Agents should communicate through typed artifacts and references, not by copying every prior message. The coordinator stores the canonical plan, evidence, decisions, and artifacts. For each specialist, a context builder selects the task brief, relevant dependencies, required constraints, and a bounded evidence set. The agent returns a structured result that is stored and indexed; later agents receive only the portions needed for their work.

Large shared documents are chunked and retrieved on demand. Summaries are derived and linked to source artifacts. Global decisions and immutable constraints are pinned in structured form. Context budgets are allocated by category, with deduplication and diversity controls. When an agent needs more detail, it calls an artifact retrieval tool rather than requesting the entire multi-agent history.

Architecture and implementation

- Define compact handoff schemas with artifact IDs, claims, open questions, and required next action.
- Use per-agent context policies based on role, permissions, and active task.
- Store raw artifacts externally and support scoped retrieval and source drill-down.
- Deduplicate repeated evidence and preserve only current versions plus relevant history.
- Measure context contribution and remove content that does not improve task performance.

Bottlenecks, risks, and safeguards

- Over-compression can remove a critical constraint. Pin and verify high-impact fields.
- Passing raw agent prose can propagate hallucinations as facts. Share claims with evidence and status.
- Summaries can drift across handoffs. Regenerate from canonical artifacts and validate.

Role-based filtering can hide information needed to detect a conflict. Define explicit cross-role escalation.

Validation and operating metrics

Tokens per handoff, context growth per stage, and task quality versus context size.

Critical-information omission and stale-artifact use.

Artifact retrieval frequency and latency.

Hallucination propagation across agent boundaries.

What the interviewer is testing: Whether you can manage multi-agent context as governed data flow rather than an ever-growing conversation.

57. How would you assign subtasks to agents based on capability, availability, cost, and expected quality?

This tests capability registries, scheduling, and adaptive routing under constraints.

Model answer

Each agent or worker profile should declare capabilities, supported tools, permissions, model routes, historical quality by task type, latency distribution, cost, capacity, and region. The scheduler matches task requirements to eligible profiles, filters by policy, and scores candidates using expected utility: predicted success or quality minus latency, cost, and risk penalties. Hard constraints such as data residency or required authority are applied before optimization.

For parallelizable work, the scheduler considers queue depth and critical-path impact. High-value or high-risk tasks may use a stronger specialist or redundant review, while routine tasks use cheaper workers. Online results update routing estimates, but exploration is bounded so production traffic is not used recklessly. Assignments and model routes are recorded for counterfactual evaluation.

Architecture and implementation

Maintain a versioned capability and performance registry for agents and models.

Represent task requirements and risk in a structured scheduling request.

Use eligibility filters followed by a cost-quality-latency objective.

Apply quotas, locality, and load-aware admission control.

Canary new specialists and use contextual-bandit or rules-based exploration only within safe task classes.

Bottlenecks, risks, and safeguards

Historical averages hide performance differences by domain or input complexity.

Routing to the cheapest available agent can increase retries and total cost.

A self-reported capability is not evidence. Validate through tests and production outcomes.

Adaptive routing can create feedback loops in which underused agents receive no data. Preserve controlled exploration.

Validation and operating metrics

Routing regret, assignment success, and quality by task-agent pair.

Queue delay, utilization, cost, and critical-path latency.

Fallback and reassignment rate.

Fairness and stability of exploration across eligible routes.

What the interviewer is testing: Whether you can schedule agent work as a constrained optimization problem using measured capability rather than role names.

58. One specialized agent becomes unavailable during execution. How should the system reassign or degrade the task?

The answer should preserve contract compatibility, state, and quality expectations during failover.

Model answer

The failed specialist should not own unique state. Its assignment, input artifact versions, partial outputs, and checkpoints are stored centrally. The coordinator marks the lease expired, classifies whether the step was read-only or may have side effects, and reconciles any in-flight operations. It then consults the capability registry for an eligible substitute that satisfies permissions, region, and output contract.

If a validated substitute exists, the step is reassigned with the same task ID and remaining budget. If only a lower-quality route exists, the workflow may reduce scope, add extra validation, or request approval. If no safe substitute exists, the task pauses or returns a partial result. Failover should not silently change a legal, security, or data-residency property.

Architecture and implementation

- Use durable assignments with leases and heartbeats.
- Store all inputs and partial artifacts outside the worker process.
- Maintain compatibility and quality tiers for substitute agents.
- Reconcile side-effect status before replaying work.
- Define degradation policies per task class, including additional review or explicit user notice.

Bottlenecks, risks, and safeguards

- Reassigning a task with a new operation ID can duplicate actions.
- A fallback may lack access to required data or violate regional policy.
- Partial outputs from the failed agent may be corrupt or incomplete. Validate before reuse.
- Frequent reassignment can cause thrashing. Use circuit breakers and stable capacity reservations.

Validation and operating metrics

- Failover success, time to reassignment, and work lost.
- Quality delta and added validation cost under degradation.
- Duplicate operations or policy violations during failover.
- Specialist availability, lease expiry, and circuit-breaker activity.

What the interviewer is testing: Whether you can fail over stateful agent work without sacrificing side-effect safety or policy constraints.

59. How would you evaluate whether adding more agents improves task performance rather than only increasing cost and latency?

This tests experimental design and attribution for architecture complexity.

Model answer

I would compare controlled system variants on the same representative task distribution: single agent, single agent plus deterministic validators, and the proposed multi-agent design. Metrics must include final task success, claim or action correctness, path efficiency, latency, cost, safety, and recoverability. I would

segment by task type because multi-agent benefits may exist only for parallel research or specialist domains.

Ablations reveal which role contributes value. Remove the reviewer, merge researcher and analyst, or replace a specialist with a tool. Inspect whether quality gains come from more compute rather than specialization by giving the single-agent baseline a comparable token budget. Online canaries should monitor user outcomes and operational burden, not just offline judge scores.

Architecture and implementation

- Create matched task sets with clear outcomes and difficult cases that motivate specialization.
- Run ablations and compute-matched baselines.
- Measure communication overhead, duplicate work, conflict resolution, and failure isolation.
- Use confidence intervals and significance or practical-effect thresholds for deployment decisions.
- Retain the simplest architecture for task segments without meaningful multi-agent lift.

Bottlenecks, risks, and safeguards

- LLM judges may prefer polished multi-agent outputs even when facts are unchanged.
- Average metrics can hide severe tail failures and coordination loops.
- Giving the multi-agent system more tools or evidence makes the comparison unfair.
- A small quality lift may not justify production complexity or incident surface.

Validation and operating metrics

- Incremental success and error reduction per added dollar and second.
- Role-level marginal contribution from ablations.
- Coordination tokens, handoff failures, and operational incidents.
- Task-segment break-even points for cost and quality.

What the interviewer is testing: Whether you can prove architectural value with fair baselines, ablations, and total-system metrics.

60. How would you prevent one compromised agent from influencing every other agent in the system?

This is a trust-boundary and blast-radius question for multi-agent systems.

Model answer

Agents should not trust one another as authorities. Every inter-agent message is treated as untrusted input and must conform to a schema, include provenance, and pass policy and content checks. Agents have least-privilege identities, separate tool scopes, and no ability to delegate credentials. Shared artifacts are immutable or versioned, signed by the platform, and labeled by source and trust level.

The coordinator controls task assignment and validates outputs before they become inputs to high-risk steps. Critical actions require independent evidence or human approval, not merely another agent recommendation. Network egress, data access, and tool calls are enforced at the gateway. If compromise is suspected, the platform can revoke the agent identity, quarantine its artifacts, trace dependent decisions, and replay affected work with a clean component.

Architecture and implementation

- Use per-agent service identities and operation-level authorization.
- Route all messages through a broker or coordinator with schema, size, and policy enforcement.
- Tag artifacts with provenance, sensitivity, trust, and validation status.

Prevent agents from modifying another agent's outputs; create new versions and review findings instead.

Maintain dependency lineage so contaminated artifacts and downstream decisions can be invalidated.

Bottlenecks, risks, and safeguards

Prompt injection can propagate through apparently helpful summaries. Preserve untrusted-content labels across handoffs.

A shared memory store can become a poisoning channel. Require write permissions, moderation, and provenance.

Using the same credentials for all agents defeats isolation.

A compromised coordinator has broad impact. Minimize its privileges and protect policy services independently.

Validation and operating metrics

Unauthorized inter-agent requests and policy denials.

Artifact validation coverage and contamination propagation depth.

Time to revoke, quarantine, and replay affected work.

Results of compromise simulations and cross-agent prompt-injection tests.

What the interviewer is testing: Whether you design multi-agent collaboration as a zero-trust distributed system with containment and lineage.

SECTION 07

Human-in-the-Loop and Authorization

These scenarios test approval boundaries, review usability, authorization propagation, approval integrity, escalation, and handling of conflicting instructions.

Section focus: Human review must be risk-based and bound to an exact action. Authorization is enforced by trusted services at execution time, never inferred from conversational authority.

61. A procurement agent can research vendors, negotiate terms, and create purchase orders. At which points should human approval be mandatory?

The interviewer expects a risk-tiered workflow with separation of duties and exact-action approval.

Model answer

I would allow autonomous collection of public vendor information, comparison against approved criteria, and preparation of a recommendation. Access to confidential bids or supplier records remains permission controlled. Negotiation messages can be drafted automatically, but external commitments, disclosure of non-public terms, or material deviations from approved playbooks require review. Creating or releasing a purchase order requires approval based on amount, category, budget ownership, supplier risk, and policy.

The agent prepares a complete action package: selected vendor, evaluated alternatives, price and terms, budget code, policy checks, risk flags, and the exact purchase-order payload. The approver reviews that package, and approval is bound to its digest. The agent that proposes the purchase cannot approve it. Changes after approval invalidate the authorization.

Architecture and implementation

- Classify actions into research, internal recommendation, external communication, commercial commitment, and financial execution.
- Map amount, supplier status, contract deviation, data disclosure, and regulated category to approval levels.
- Use segregation of duties among proposer, approver, and payment executor.
- Create an immutable approval record linked to exact terms, attachments, recipient, and expiration.
- Use post-execution reconciliation against the procurement and financial systems.

Bottlenecks, risks, and safeguards

- A draft message may still create apparent commitment if sent externally. Treat sending and drafting as separate actions.
- Breaking a purchase into smaller orders can bypass thresholds. Aggregate related spend and intent.
- Approvers may not see hidden contract deviations. Generate a deterministic redline and policy comparison.
- Supplier data may change between approval and execution. Revalidate material fields immediately before commit.

Validation and operating metrics

- Approval coverage and unauthorized commitment rate.
- Straight-through research and recommendation time.
- Approval turnaround, rejection reasons, and post-approval change frequency.
- Duplicate purchase orders, budget exceptions, and reconciliation mismatches.

What the interviewer is testing: Whether you can place human judgment at commitment boundaries

and enforce separation of duties and approval integrity.

62. How would you distinguish actions that require confirmation from those that can be executed automatically?

The expected answer uses consequence, reversibility, confidence, authorization, and user expectations.

Model answer

I would evaluate the action by impact, reversibility, scope, data sensitivity, external visibility, financial or legal consequence, confidence, and whether it matches a standing user preference. Read-only lookups and reversible low-impact changes within explicit standing authority can be automatic. Irreversible, externally visible, privileged, high-value, or novel actions require confirmation or approval.

The decision is policy-driven. The model can classify intent and risk signals, but a policy engine makes the final determination. Confirmation should show the exact action and important consequences, not a vague question such as "Proceed?" The system can learn user convenience preferences only within organizational and safety limits; users cannot waive controls they are not authorized to waive.

Architecture and implementation

- Create an action-risk matrix with static operation metadata and dynamic parameters such as amount or audience.
- Support standing approvals with scope, maximum impact, expiration, and revocation.
- Bind confirmation to recipient, payload, resource, amount, policy version, and time.
- Add step-up authentication for sensitive actions.
- Use post-condition checks and notifications for automatically executed reversible actions.

Bottlenecks, risks, and safeguards

- Too many confirmations create habituation and approval fatigue.
- A low-risk action repeated many times can become high impact. Consider cumulative session and period risk.
- User preference cannot override enterprise policy or role permissions.
- The model may understate consequence. Determine risk from tool and domain metadata.

Validation and operating metrics

- Confirmation rate by action class and user abandonment.
- Unauthorized or regretted automatic-action rate.
- Approval override and reversal rate.
- Cumulative-risk triggers and accuracy of the risk classifier.

What the interviewer is testing: Whether you can balance usability with risk using enforceable policy rather than asking for confirmation before everything.

63. An agent proposes an action and receives approval, but the underlying data changes before execution. Is the approval still valid?

This tests time-of-check versus time-of-use and approval binding.

Model answer

Approval is valid only for the exact material facts the reviewer assessed. The approval record should include a digest of the action payload, referenced data versions, policy version, relevant calculations, and expiration time. Immediately before execution, the gateway re-resolves canonical entities and compares current material fields with the approved snapshot.

Non-material changes, such as a refreshed descriptive label, may be allowed under policy. Material changes - amount, recipient, permissions, inventory, contract terms, eligibility, or risk status - invalidate approval and return the action for review. The agent must not silently update the proposal and reuse the old approval. For rapidly changing data, use shorter approval validity or reserve resources during approval.

Architecture and implementation

- Hash the canonical action and relevant source versions into the approval record.
- Define materiality rules per action type and compare them at commit time.
- Use optimistic concurrency, version checks, or reservations for mutable resources.
- Expire approvals and require reauthentication for high-risk delays.
- Display a concise change diff when reapproval is needed.

Bottlenecks, risks, and safeguards

- A broad text approval such as "buy it" cannot prove what was reviewed.
- Minor changes can be used to hide a material one. Compare structured fields and derived totals.
- Reservations may expire or create resource contention. Surface reservation status and deadlines.
- A policy change can make an approved action newly prohibited. Reevaluate current policy before execution.

Validation and operating metrics

- Approval invalidation rate and material changes detected before commit.
- Time between approval and execution and resulting reapproval burden.
- Incidents from stale approvals or concurrency conflicts.
- False invalidations caused by overly sensitive materiality rules.

What the interviewer is testing: Whether you understand approval as a versioned authorization over exact facts, not a durable conversational permission.

64. How would you present an agent’s proposed action to a human reviewer so that the reviewer can make an informed decision?

The challenge is designing a review artifact that supports real judgment instead of rubber-stamping.

Model answer

The reviewer should see the goal, proposed action in plain language, affected entities, material fields, expected outcome, alternatives considered, evidence, policy checks, uncertainty, and irreversible consequences. The interface should highlight exceptions and changes from defaults rather than burying them in generated prose. For a communication, show the exact recipient, subject, attachments, and final body. For a transaction, show amount, currency, account, fees, and post-action state.

The review should also explain why approval is required and what authority the reviewer is exercising. Source links and tool receipts allow drill-down. The reviewer can approve, reject, edit within controlled fields, or request more evidence. Any edit creates a new action version and re-runs validation. The interface records the exact content reviewed and the decision rationale.

Architecture and implementation

- Use domain-specific review templates generated from structured action data.
- Prioritize exceptions, deltas, risk flags, and missing evidence above background detail.
- Provide expandable provenance rather than forcing reviewers to read the full agent trace.
- Show the exact commit action and bind approval to it.
- Support accessible, mobile-safe review for time-sensitive workflows without reducing material information.

Bottlenecks, risks, and safeguards

- Long model explanations create cognitive overload. Use concise structured summaries with drill-down.
- Hiding uncertainty or failed checks creates false confidence.
- Allowing free-form edits can bypass validation. Rebuild and revalidate the action after edits.
- The interface may imply that the agent is more certain than the evidence supports. Display evidence quality and unresolved conflicts.

Validation and operating metrics

- Reviewer decision time, rejection and correction rates, and post-approval reversals.
- Comprehension testing and frequency of missed risk flags.
- Approval rationale completeness and auditability.
- False approvals and unnecessary escalations by review-template version.

What the interviewer is testing: Whether you can design human review as an effective control with the right information density and exact action binding.

65. An agent requests approval too frequently, causing users to approve actions without reading them. How would you reduce approval fatigue?

The expected answer improves risk classification and review design without weakening required controls.

Model answer

I would analyze approval traces by action type, risk, reviewer decision, edits, reversals, and incidents. Low-risk actions that are almost always approved unchanged may be candidates for bounded standing authority, batching, or post-action notification. High-risk decisions remain individually approved. Repeated related actions can be presented as one clearly scoped batch with aggregate impact and item-level exceptions.

The system should ask only when user input changes the outcome. Missing fields that can be resolved from authoritative data should not trigger approval. Review screens should emphasize material differences and exceptions. Standing approvals include limits, scope, expiration, and easy revocation. Any reduction in prompts should be canaried and evaluated for regret and incident rate.

Architecture and implementation

- Segment approvals by risk and observed reviewer value, not only by volume.
- Introduce bounded standing approvals and auto-execution for reversible, low-impact, high-confidence cases.
- Batch homogeneous actions while preserving individual visibility and the ability to exclude items.

Use exception-based review for policy deviations or changed facts.

Notify users after automatic actions and provide a simple reversal path where feasible.

Bottlenecks, risks, and safeguards

Historical approval rate may reflect inattentive reviewers, not low risk.

Batch approval can hide one dangerous item. Highlight outliers and enforce aggregate thresholds.

Standing authority can outlive user intent. Expire and periodically reconfirm it.

Reducing prompts without monitoring reversals can silently increase harm.

Validation and operating metrics

Approvals per task and reviewer time spent.

Read or interaction signals, correction rate, and post-approval regret.

Incident and reversal rate after automation changes.

Percentage of approvals that materially alter the proposed action.

What the interviewer is testing: Whether you can improve human control quality by reducing low-value prompts rather than simply removing approvals.

66. How would you prevent an agent from splitting a high-risk action into smaller actions to bypass approval thresholds?

This tests cumulative-risk analysis and intent-level policy enforcement.

Model answer

Policy must evaluate the business intent and aggregate effect, not each API call independently. The orchestrator groups related actions by user, tenant, beneficiary, resource, time window, task, and semantic goal. It computes cumulative amount, data volume, recipient exposure, privilege change, or other risk dimensions. A plan validator detects when several low-level steps produce a controlled high-impact outcome.

Approval attaches to the aggregate plan or commitment, and the tool gateway enforces remaining authorized capacity. If the agent modifies the plan or attempts an unapproved related action, execution is blocked. Historical activity outside the current run can also count toward thresholds, such as daily transfer or email-recipient limits.

Architecture and implementation

Assign an intent or campaign ID to related actions and carry it through tool calls.

Maintain cumulative counters and risk state across steps, sessions, and time windows.

Analyze the plan graph for composed effects, including privilege escalation and data reconstruction.

Bind approval to aggregate limits and decrement authorized capacity at execution.

Alert on repeated near-threshold actions and unusual decomposition patterns.

Bottlenecks, risks, and safeguards

Simple amount aggregation misses semantic composition, such as combining datasets to reveal sensitive facts.

Attackers may spread actions across sessions or identities. Apply organizational and beneficiary-level controls where lawful.

Over-aggregation can block legitimate independent work. Use task, purpose, and entity context.

The model cannot be trusted to label intent honestly. Derive grouping from state and observed actions.

Validation and operating metrics

- Threshold-bypass attempts detected and blocked.
- False positives from aggregation policy.
- Cumulative-risk incidents and near-threshold patterns.
- Approval coverage for composed high-impact outcomes.

What the interviewer is testing: Whether you enforce policy over cumulative real-world effect rather than one isolated tool call.

67. A user approves sending an email, but the agent modifies the recipient or message afterward. How would you bind approval to the exact action?

The answer should describe canonicalization, hashing, expiration, and revalidation.

Model answer

Before approval, the system creates a canonical email action containing normalized recipients, subject, exact body, attachments and hashes, sender identity, scheduled time, and any tracking or confidentiality settings. It computes an action digest and stores it with the approval identity, timestamp, authentication strength, and expiration. The send tool accepts only that approved action ID and verifies that the digest matches the payload.

Any modification creates a new version and invalidates the prior approval. Even seemingly minor changes are classified for materiality, but recipient, attachment, and meaning-changing body edits always require reapproval. The model does not receive a general permission token to send arbitrary mail; the gateway grants one-time authority for the exact action.

Architecture and implementation

- Canonicalize addresses, aliases, attachment versions, and content before hashing.
- Store immutable action versions and one-time approval tokens.
- Verify digest, approver authority, expiration, and current policy at send time.
- Mark the action consumed after successful submission to prevent reuse.
- Record the provider message ID and reconcile submission status.

Bottlenecks, risks, and safeguards

- Display text can differ from canonical addresses. Show and hash canonical recipients.
- Remote attachment links can change after approval. Snapshot or hash attachment content.
- A one-time token can be stolen if exposed to the model. Keep it inside trusted execution services.
- Approval replay can duplicate a send. Combine consumption state with an idempotency key.

Validation and operating metrics

- Payload-digest mismatch and expired-approval blocks.
- Emails sent without exact approval, expected to be zero where required.
- Reapproval frequency by change type.
- Duplicate-send and post-send discrepancy rates.

What the interviewer is testing: Whether you can implement cryptographically or structurally exact approval rather than conversational consent.

68. How would you implement role-based and attribute-based access control for agent tools?

The expected answer propagates identity to the tool layer and combines stable roles with contextual attributes.

Model answer

RBAC provides coarse entitlements such as support agent, finance approver, or developer. ABAC refines each decision using tenant, resource owner, region, data classification, environment, amount, device posture, purpose, and time. The authenticated user and agent runtime have separate identities; the agent acts on behalf of the user but receives only scoped delegated authority. The model never creates or edits these claims.

At each tool call, the gateway sends a structured authorization request to a policy engine: actor, delegated user, operation, resource, context, and proposed parameters. The decision can allow, deny, require approval, redact fields, or limit scope. Downstream systems still enforce their own access controls. Every decision and policy version is logged.

Architecture and implementation

- Use a central identity provider and short-lived, audience-bound tokens.
- Define operation and resource attributes in the tool registry and canonical resource service.
- Evaluate policy at planning for feasibility and again at execution for current state.
- Use field- and row-level filtering for data-returning tools.
- Test policies with positive, negative, cross-tenant, and privilege-escalation cases.

Bottlenecks, risks, and safeguards

- Relying only on the application router permits direct tool-gateway bypass.
- Broad service credentials erase user-level accountability. Use delegated scopes and downstream checks.
- Attributes derived from model text can be forged. Resolve them from trusted systems.
- Policy changes can affect in-flight approvals. Re-evaluate current policy at commit time.

Validation and operating metrics

- Authorization allow, deny, and step-up rates by tool and policy.
- Least-privilege coverage and unused broad permissions.
- Cross-tenant and escalation test results.
- Policy evaluation latency and availability.

What the interviewer is testing: Whether you can enforce user and context-aware least privilege through trusted identity and policy infrastructure.

69. A manager asks an agent to perform an action that the manager is not personally authorized to perform. Should the agent execute it?

The correct principle is that conversational seniority cannot create authority.

Model answer

No. The agent can act only within the authenticated caller's delegated authority and applicable organizational policy. A job title or claim in conversation is not an authorization credential. The tool gateway resolves the caller's role and resource permissions from trusted identity systems and denies the action if they are insufficient.

The agent should explain the restriction without disclosing sensitive policy internals, and offer an authorized path: request approval from the correct role, create a ticket, prepare the action for an authorized operator, or provide the required evidence. It must not use its own service account to bypass the manager's limits. Emergency or break-glass access, if the organization supports it, is an explicit audited workflow with stronger authentication and retrospective review.

Architecture and implementation

- Separate identity, role, and delegation from natural-language instructions.
- Use policy-engine decisions at the tool gateway and downstream resource.
- Provide approval and delegation workflows with scope and expiration.
- Implement break-glass procedures outside normal agent discretion.
- Log denied attempts and any subsequent authorized execution as linked events.

Bottlenecks, risks, and safeguards

- A privileged agent service account can become an authority bypass.
- Social engineering may pressure the system to treat urgency as permission.
- Overly detailed denial messages can reveal organizational structure or sensitive access rules.
- A delegate may have broad standing authority; ensure the delegated action is still scoped and current.

Validation and operating metrics

- Unauthorized execution rate and denial accuracy.
- Approval-path completion and time to authorized resolution.
- Break-glass usage, review completion, and incidents.
- Attempts to invoke tools outside delegated user scope.

What the interviewer is testing: Whether you maintain authorization integrity under social pressure and offer safe escalation rather than bypass.

70. How should an agent handle conflicting instructions from the user, organization policy, system rules, and retrieved content?

This tests instruction hierarchy, trusted versus untrusted data, and transparent resolution.

Model answer

The system should encode an instruction hierarchy in trusted control logic. Platform and safety policy constrain all behavior. Organization policy and the authenticated user's permissions constrain allowed actions. The user's current request controls within those boundaries. Retrieved documents, web pages, emails, and tool outputs are evidence, not instructions to the agent unless an approved workflow explicitly treats a trusted field as a command.

When instructions conflict, the orchestrator blocks prohibited actions and the response explains the applicable constraint at an appropriate level. If organization policy documents themselves conflict, the system uses authority, effective date, jurisdiction, and policy ownership to resolve them or escalates. Prompt wording alone should not implement the hierarchy; policy engines, content labeling, and tool authorization enforce it.

Architecture and implementation

- Label every context segment by source, trust level, purpose, and authority.
- Keep system and policy instructions in protected channels and external content in data fields.
- Use a policy engine for action constraints and a document-authority model for retrieved rules.

Detect and ignore instructions embedded in untrusted content.

Return a clear refusal, clarification, or escalation when conflict cannot be safely resolved.

Bottlenecks, risks, and safeguards

Retrieved text may mimic system instructions. Formatting is not a trust signal.

Organization policy can be outdated or jurisdiction-specific. Track version and applicability.

The model may follow the most recent or forceful text instead of the most authoritative source.

An overly broad refusal can block legitimate work. Separate prohibited action from allowed assistance.

Validation and operating metrics

Policy-conflict resolution accuracy and unsafe instruction-following rate.

Indirect prompt-injection success under adversarial tests.

False refusal and safe-alternative completion rates.

Unresolved policy conflicts routed to owners and resolution time.

What the interviewer is testing: Whether you can enforce a real trust hierarchy and treat retrieved content as untrusted data rather than model control text.

SECTION 08

Security, Safety, and Trust Boundaries

These questions cover indirect prompt injection, data exfiltration, code and database sandboxing, credential protection, tool-output poisoning, and incident containment.

Section focus: Assume user content, retrieved documents, websites, tool output, and peer-agent messages are untrusted. Security controls must exist outside the model and remain effective even when the model is manipulated.

71. A browser agent reads a webpage containing instructions to upload internal documents. How would you prevent indirect prompt injection?

The interviewer is testing whether the candidate treats web content as hostile data and constrains tool authority.

Model answer

The page content must be labeled as untrusted evidence and never merged into the instruction channel. The browser or extraction layer should isolate visible content, links, forms, scripts, and metadata, and remove or flag instruction-like text. The model receives a task-specific data representation with a reminder that page instructions have no authority. However, prompting is only one layer: the agent should not possess a general upload capability or broad file access during a browsing task.

Every external action passes through a policy-enforced tool gateway. Uploading or transmitting internal data requires explicit user intent, destination validation, data classification checks, and usually human approval. The gateway blocks destinations outside approved domains and applies DLP inspection. A compromised model therefore cannot convert a page instruction into an authorized data transfer.

Architecture and implementation

- Separate trusted control instructions from untrusted page content in the context data model.
- Use a browser sandbox with restricted network, downloads, filesystem, clipboard, and credential access.
- Grant task-specific capabilities; a read-only research agent should not have file-upload or email tools.
- Apply destination allowlists, content classification, DLP, and exact-action approval to outbound transfers.
- Preserve source labels through summaries and inter-agent handoffs.

Bottlenecks, risks, and safeguards

- Injection may be hidden in alt text, metadata, code blocks, images, or tool-generated summaries.
- A model can encode data into URLs or search queries. Inspect all egress channels, not only uploads.
- Content sanitization alone is incomplete because legitimate text can still manipulate the model.
- Whitelisted domains can host user-generated content. Validate endpoint, operation, and data purpose.

Validation and operating metrics

- Prompt-injection attack success under adversarial web tests.
- Blocked egress attempts, DLP findings, and unauthorized tool requests.
- False positives that prevent legitimate browsing or submissions.
- Coverage of source trust labels across the execution trace.

What the interviewer is testing: Whether you use defense in depth - content isolation, least privilege,

policy enforcement, and egress control - rather than a stronger prompt alone.

72. An agent can read internal files and send external emails. How would you prevent data exfiltration?

This tests capability separation, information-flow control, and outbound policy.

Model answer

I would avoid granting both unrestricted read and unrestricted external send to the same execution context. File access is scoped by user permissions, task purpose, and data classification. The email capability is scoped to approved recipients and requires a prepared message package. Before sending, a trusted service evaluates whether the content is derived from restricted sources, applies DLP and secret detection, and enforces organization rules for external domains and attachments.

For sensitive workflows, use two-stage control: the agent can draft from approved evidence, but a human reviews the exact recipient, body, and attachments. Data lineage tags follow extracted text and summaries so the system can detect that seemingly paraphrased content came from restricted files. Network egress and tool logs provide an independent control even if the model is compromised.

Architecture and implementation

- Use separate read and send capabilities with task-scoped delegated credentials.
- Propagate data classification and provenance through artifacts and generated drafts.
- Apply recipient policy, domain allowlists, DLP, secret scanning, and attachment inspection at the send gateway.
- Require exact-action approval for external communication containing sensitive or novel data.
- Rate-limit and monitor unusual recipient counts, encoded payloads, and repeated small transfers.

Bottlenecks, risks, and safeguards

- Paraphrasing can bypass naive string-based DLP. Combine classifiers, lineage, and policy.
- Search queries, URLs, or tool arguments can become covert exfiltration channels.
- Internal recipients can forward data externally. Apply policy appropriate to the real destination and sensitivity.
- Broad service accounts can read more than the user. Enforce user-level and purpose-level access.

Validation and operating metrics

- Unauthorized outbound attempts blocked and confirmed exfiltration incidents.
- DLP precision and recall on agent-generated content.
- Sensitive-data lineage coverage from source to outbound action.
- External-send approval, rejection, and post-send incident rates.

What the interviewer is testing: Whether you reason about data flow across combined capabilities and enforce controls at egress and identity layers.

73. How would you separate trusted instructions, user content, retrieved data, tool output, and external web content?

The answer should describe typed context segments and policy decisions based on origin and authority.

Model answer

I would represent context as typed segments with origin, trust level, authority, sensitivity, and allowed use. System and platform policy are trusted control instructions. Organization configuration is trusted within its scope and version. User messages express intent but cannot override policy or grant permissions. Retrieved documents, tool results, peer-agent outputs, and web pages are data that may contain claims or malicious instructions; they do not control the agent.

The context builder preserves these boundaries when assembling prompts, using explicit delimiters and structured content fields. More importantly, the orchestrator and tool gateway enforce policy independently of model interpretation. Summarization retains source labels, and any command-like text from untrusted content is quoted or represented as content. Actions are authorized from trusted identity and state, not from statements inside retrieved text.

Architecture and implementation

- Use a context schema with source ID, trust class, purpose, permissions, and provenance for each segment.
- Keep instructions and data in separate model message or content channels where supported.
- Apply content sanitization and injection detection without treating them as the only defense.
- Prevent untrusted content from adding tools, changing budgets, or altering authorization state.
- Validate generated actions against trusted task state and policy before execution.

Bottlenecks, risks, and safeguards

- Delimiters alone do not guarantee the model will ignore malicious text.
- A trusted internal document can still be outdated or compromised. Authority and integrity are distinct.
- Summaries may lose trust labels and turn quoted instructions into apparent commands.
- Tool output may include attacker-controlled fields even from a trusted API.

Validation and operating metrics

- Trust-label propagation and context-segmentation coverage.
- Instruction-following rate from untrusted sources under tests.
- Actions blocked because they conflict with trusted state or policy.
- Source-attribution and provenance completeness in generated outputs.

What the interviewer is testing: Whether you can encode trust as data and enforce authority outside natural-language prompt hierarchy.

74. A malicious document contains text designed to influence the agent. At what stages should the document be treated as untrusted?

The correct answer is throughout the lifecycle, including derived artifacts.

Model answer

The document is untrusted from ingestion through deletion. The ingestion service scans file type, malware, macros, embedded objects, links, and metadata in a sandbox. Parsing and OCR run with restricted

privileges. Extracted text retains source and trust labels. Chunking, embedding, indexing, retrieval, summarization, and generation must not upgrade the content to trusted instruction status.

When retrieved, the document is presented as evidence only. Tool calls proposed because of its content face the same policy and authorization checks as any other action. Derived summaries and claims remain linked to the untrusted source and require validation before use in consequential decisions. If the source is later identified as poisoned, lineage allows affected indexes, memories, outputs, and decisions to be quarantined or invalidated.

Architecture and implementation

- Sandbox file parsing and disable active content, external fetches, and unsafe file handlers.
- Tag document, chunks, embeddings, summaries, and claims with source trust and integrity status.
- Use content-based access controls and quarantine suspicious sources.
- Require independent support or source-authority checks for high-impact claims.
- Maintain lineage from document versions to downstream artifacts and decisions.

Bottlenecks, risks, and safeguards

- OCR or image captions can reveal hidden injection not present in extracted text.
- A summary can launder malicious instructions into a trusted-looking artifact.
- Embedding retrieval may surface poisoned content more often because it is optimized for target queries.
- Deleting the source without invalidating derived content leaves contamination active.

Validation and operating metrics

- Malicious-document detection, quarantine, and false-positive rates.
- Trust-label retention through derived artifacts.
- Poisoned-source influence on tool selection or final answers.
- Time to identify and invalidate downstream dependencies after a source incident.

What the interviewer is testing: Whether you treat untrusted content as an end-to-end information-security property rather than an ingestion-only problem.

75. How would you sandbox an agent that can generate and execute Python code?

This tests process isolation, resource limits, artifact handling, and separation from production credentials.

Model answer

I would run code in an ephemeral, non-privileged container or microVM created for the task, with a read-only base image, an empty or explicitly mounted workspace, no host mounts, no container socket, and a minimal allowlisted runtime. Network access is disabled by default or proxied through an allowlist. Credentials are not placed in the environment. Input datasets are copied into scoped storage, and outputs are exported only through a controlled artifact service.

The execution service enforces CPU, memory, disk, process, file, and wall-time limits, kills descendants, records the image and package lock, and captures stdout, stderr, exit code, and produced files. Generated code is scanned for prohibited operations before execution, but sandboxing remains the primary defense. High-risk packages or native extensions are blocked or run in stronger isolation.

Architecture and implementation

- Use per-run containers or microVMs with unprivileged users, seccomp, capability dropping, and read-only roots.

Provide a curated package set or isolated dependency install through an approved mirror.

Disable network by default and mediate any allowed outbound request through policy and logging.

Apply quotas and terminate fork bombs, infinite loops, large writes, and excessive output.

Scan and classify exported artifacts before making them available to users or other agents.

Bottlenecks, risks, and safeguards

Container isolation alone may be insufficient for hostile native code. Use stronger virtualization for high-risk workloads.

Package installation can execute arbitrary setup code or introduce supply-chain attacks.

Generated files can contain malware, formulas, or active content.

Secrets can leak through mounted datasets, logs, error traces, or network metadata.

Validation and operating metrics

Sandbox escape attempts, policy denials, and suspicious process behavior.

Resource-limit terminations and execution success by task type.

Artifact scan findings and blocked exports.

Image reproducibility, patch age, and dependency vulnerability status.

What the interviewer is testing: Whether you can safely execute untrusted code with strong isolation and controlled inputs, outputs, and dependencies.

76. What filesystem, network, package, CPU, memory, runtime, and credential restrictions would you impose on a code-execution agent?

The interviewer wants concrete controls across every resource boundary.

Model answer

Filesystem access should be limited to a per-task workspace, with the base image read-only, no device files, no host paths, no shared home directory, path traversal controls, file-count and disk quotas, and controlled artifact export. Network should be denied by default; if needed, it should use an authenticated proxy with DNS and destination allowlists, method and volume limits, and full egress logging. Packages should come from a pinned allowlisted repository with hashes and vulnerability scanning.

CPU, memory, processes, threads, GPU, and wall time need hard quotas, plus output-size and log limits. The runtime should drop Linux capabilities, use an unprivileged UID, seccomp and AppArmor or equivalent, and terminate child processes on timeout. Credentials should be absent by default and, when unavoidable, be short-lived, audience-bound, operation-scoped tokens injected into a trusted sidecar rather than exposed directly to code.

Architecture and implementation

Set read-only root, scoped writable volume, disk and inode quota, and artifact allowlist.

Deny network or mediate it through a policy proxy with domain, protocol, and byte limits.

Pin images and packages by digest and block arbitrary install scripts or native builds where possible.

Enforce cgroups or equivalent for CPU, memory, PID, GPU, and time limits.

Use brokered credentials and revoke them when the operation or sandbox ends.

Bottlenecks, risks, and safeguards

DNS rebinding and redirects can bypass simple domain allowlists. Validate resolved destinations and every redirect.

- Output channels and timing can exfiltrate data even without general network access. Bound and inspect outputs.
- A dependency may be compromised after approval. Pin content hashes and continuously rescan.
- A shared cache can leak data across runs. Namespace or avoid sensitive caches.

Validation and operating metrics

- Resource-violation and network-denial events.
- Unpinned dependency usage and vulnerability findings.
- Credential scope, lifetime, and unauthorized-use attempts.
- Cross-run isolation and sandbox-escape test results.

What the interviewer is testing: Whether you can enumerate enforceable sandbox controls and understand that credentials and egress are as important as CPU limits.

77. How would you protect API keys and credentials used by agent tools from being exposed to the model?

The expected design uses brokered execution and short-lived credentials outside model context.

Model answer

The model should never see raw long-lived credentials. It emits a structured request naming an approved capability. A trusted tool gateway authenticates the run, authorizes the requested operation, obtains or mints a short-lived scoped token from a secret manager or identity broker, and calls the downstream service. Responses are filtered before returning to the model so headers, tokens, and sensitive diagnostic data are removed.

Credentials are audience-bound, tenant-bound, environment-bound, and limited to the operation where possible. Logs and traces use allowlisted fields and automatic secret redaction. Tool schemas and error messages must not reveal secret locations. Rotation and revocation happen centrally without changing prompts or agent code.

Architecture and implementation

- Use workload identity and secret managers instead of static keys in prompts, code, or environment variables.
- Broker calls through the tool gateway and inject credentials only inside the trusted connector.
- Mint short-lived, least-privilege tokens with explicit audience and operation scopes.
- Redact secrets from requests, responses, errors, traces, and generated artifacts.
- Continuously scan repositories, prompts, logs, and object storage for accidental exposure.

Bottlenecks, risks, and safeguards

- A tool may echo authorization headers or signed URLs in errors. Normalize responses at the adapter.
- A model may request a tool that can reveal secret values. Do not expose secret-read operations.
- Long-lived tokens in worker memory can be stolen by generated code. Use brokered sidecars or separate processes.
- Logs are often broader than production access. Apply access control and retention to observability data.

Validation and operating metrics

- Secret-exposure findings, token lifetime, and privilege breadth.
- Unauthorized credential use and revoked-token attempts.
- Percentage of tool calls using workload identity or brokered tokens.
- Mean time to rotate or revoke a compromised credential.

What the interviewer is testing: Whether you keep secrets entirely in trusted execution infrastructure and design for least privilege, redaction, and revocation.

78. An agent is permitted to query a database. How would you prevent destructive SQL, unauthorized joins, excessive scans, and sensitive-data disclosure?

The answer should combine data access architecture, query validation, resource governance, and result filtering.

Model answer

I would give the agent a read-only data service or query gateway rather than direct database credentials. The service exposes approved datasets, views, and semantic models and derives row- and column-level access from the authenticated user and purpose. Generated SQL is parsed into an AST, checked against an allowlist of operations and objects, and rejected if it contains writes, DDL, unsafe functions, cross-tenant references, or prohibited joins.

Before execution, the gateway uses EXPLAIN or an optimizer estimate to enforce row, byte, time, and cost limits. Queries run with statement timeout and resource groups, preferably against replicas or warehouses isolated from transactions. Results are capped, aggregated where possible, and passed through data-loss and sensitivity policies. High-risk outputs require review or are returned only as approved aggregates.

Architecture and implementation

- Use read-only, least-privilege service roles and approved views or a semantic layer.
- Parse and validate SQL structurally; do not rely on string matching.
- Apply row, column, tenant, purpose, and masking policies inside the data platform.
- Use query-cost estimation, timeouts, quotas, result limits, and workload isolation.
- Log normalized query, authorization decision, plan cost, result size, and downstream use.

Bottlenecks, risks, and safeguards

- A read-only query can still exfiltrate or infer sensitive data.
- Joining two allowed datasets can reveal a prohibited attribute. Use purpose-aware join controls and minimum group sizes.
- EXPLAIN estimates may be wrong. Enforce runtime byte and time limits too.
- Returning raw rows to the model expands privacy exposure. Prefer calculations in the data layer and minimal results.

Validation and operating metrics

- Blocked write or unauthorized-object attempts.
- Query scan bytes, runtime, timeouts, and resource impact.
- Sensitive-result policy triggers and disclosure incidents.
- Accuracy of query validation and false-rejection rate for legitimate analysis.

What the interviewer is testing: Whether you understand that safe text-to-SQL requires a governed data service, not merely a read-only database user.

79. How would you defend against tool-output poisoning, where a compromised API returns instructions intended to control the agent?

This tests zero-trust treatment of tool data and validation of both content and business semantics.

Model answer

Tool output is data, even when the API is normally trusted. The adapter validates its schema, source identity, signature or transport security, and expected semantics. Free-text fields are labeled untrusted and cannot alter the plan, policy, tools, or authorization. The context builder quotes or structures them as evidence. Any instruction-like text is ignored unless the workflow explicitly expects an approved command field from an authenticated source.

For critical tools, the system cross-checks results with an independent source, known invariants, or post-conditions. Sudden distribution changes, unexpected fields, instruction patterns, and abnormal response sizes trigger quarantine or circuit breakers. A compromised tool cannot directly call other tools; only the orchestrator can propose and authorize subsequent actions.

Architecture and implementation

- Normalize and validate tool responses at adapters before they enter agent state.
- Attach source, integrity, trust, and sensitivity metadata to every output field or artifact.
- Keep tool output in data channels and prohibit it from changing control configuration.
- Use semantic invariants, signatures, cross-source checks, and anomaly detection for important data.
- Quarantine suspicious outputs and disable affected tool versions through circuit breakers.

Bottlenecks, risks, and safeguards

- Valid schemas can still carry malicious strings or false business data.
- Summarizers may turn poisoned text into apparent instructions. Preserve trust labels through derivation.
- Multiple tools may depend on the same compromised upstream source, reducing independence.
- Overly aggressive filtering may remove legitimate instructions embedded in data; use workflow-specific parsers.

Validation and operating metrics

- Poisoning attack success and suspicious-output detection rate.
- Schema, semantic, and source-integrity failures by tool.
- Time to quarantine a tool and affected downstream artifact count.
- False positives and quality impact from output filtering.

What the interviewer is testing: Whether you maintain trust boundaries after a tool call and prevent data from becoming control simply because it came from an API.

80. An autonomous agent performs an unauthorized action in production. What information would you need for incident investigation and containment?

The expected response covers immediate containment, complete evidence, impact analysis, and durable corrective actions.

Model answer

First I would stop further harm: disable the affected workflow or tool, revoke credentials, pause related tasks, preserve evidence, and reconcile external state. I would identify the action ID, actor and delegated user, tenant, affected resource, exact payload, approval record, authorization decision, tool receipt,

downstream status, and whether similar actions are in flight. If the action is reversible, remediation follows a reviewed compensation plan.

For investigation I need the complete trace: user input, trusted instructions, retrieved content and trust labels, model and prompt versions, plan revisions, tool candidates, selected arguments, policy evaluations, state transitions, retries, errors, and final communication. I would determine whether the cause was prompt injection, permission misconfiguration, tool bug, stale approval, model error, race condition, or missing control. Then I would identify all runs with the same vulnerable path and invalidate affected artifacts or decisions.

Architecture and implementation

- Maintain immutable, access-controlled traces with correlation IDs across model, orchestrator, gateway, and downstream systems.
- Support kill switches by tool, workflow, model route, tenant, and risk class.
- Keep credential revocation and task cancellation separate from the model runtime.
- Use lineage to find related runs, source content, and downstream consequences.
- Complete a blameless postmortem with control, detection, response, and recovery improvements plus regression tests.

Bottlenecks, risks, and safeguards

- Continuing to debug before containment can increase impact.
- Logs may themselves contain sensitive information; restrict incident access and preserve chain of custody.
- Deleting or altering traces during remediation destroys evidence.
- Fixing only the prompt may leave the underlying authorization gap open.

Validation and operating metrics

- Time to detect, contain, revoke, reconcile, and recover.
- Number of affected actions, users, tenants, and downstream systems.
- Trace completeness and percentage of impacted runs identified through lineage.
- Recurrence, control-test coverage, and time to close corrective actions.

What the interviewer is testing: Whether you can operate an agent incident like a security and distributed-systems incident, with containment before explanation and evidence-driven remediation.

SECTION 09

Evaluation, Testing, and Observability

These questions test how to evaluate an agent as a trajectory through a changing environment, debug it from evidence, create adversarial suites, and deploy model changes safely.

Section focus: Score the outcome and the path. A correct answer reached through unsafe, wasteful, or irreproducible actions is not a successful agent run.

81. How would you evaluate an agent beyond checking whether its final answer is correct?

The interviewer expects multidimensional trajectory evaluation and domain-specific outcomes.

Model answer

I would evaluate the final outcome, the execution path, and the operating behavior. Outcome metrics include task completion, factual or action correctness, completeness, and user utility. Path metrics include plan validity, tool selection, argument correctness, evidence use, unnecessary steps, retries, policy compliance, and whether irreversible actions were appropriately approved. Operating metrics include latency, cost, reliability, recovery, and resource use.

The evaluation unit is a trace with a task specification and expected invariants. Some dimensions are deterministic: schema validity, permission checks, exact calculations, tool receipts, and side-effect state. Others use labeled rubrics, simulators, domain experts, or calibrated judges. I would report a scorecard rather than one average because a high-quality answer cannot compensate for a security violation.

Architecture and implementation

- Define task-level success and mandatory safety invariants before collecting outputs.
- Instrument plan, action, observation, validation, and termination events.
- Use deterministic evaluators for objective properties and human or model rubrics for semantic quality.
- Segment results by task type, risk, model route, tool, tenant, and failure stage.
- Include partial completion and correct abstention as explicit outcomes.

Bottlenecks, risks, and safeguards

- Final-answer evaluation hides unsafe or wasteful trajectories.
- A single composite score can mask catastrophic failures. Use hard gates for safety and authorization.
- Judges may reward fluent explanations over actual execution.
- Offline tasks may not reproduce real tool latency, permissions, and evolving state.

Validation and operating metrics

- Task success, correctness, completeness, abstention, and user satisfaction.
- Tool and plan accuracy, path efficiency, and unsupported-claim rate.
- Policy violations, unapproved actions, duplicate side effects, and recovery success.
- Latency, cost per successful task, and tail reliability.

What the interviewer is testing: Whether you see an agent as a decision-and-action trajectory and can separate quality from safety and efficiency.

82. What metrics would you use for task success, tool-selection accuracy, planning quality, path efficiency, latency, cost, safety, and user satisfaction?

The challenge is naming measurable definitions and avoiding vanity metrics.

Model answer

Task success should be defined from business outcomes or validated end state, with partial and abstained outcomes separated. Tool selection can use top-1 accuracy, candidate recall, no-tool-needed accuracy, and successful recovery after a wrong choice. Planning quality includes feasibility, dependency correctness, required-step coverage, revision quality, and predicted versus actual cost. Path efficiency compares validated work with redundant calls, retries, or distance from an expert or minimal safe path.

Latency should include p50, p95, and p99 end-to-end plus time by planner, model, queue, tool, and validation. Cost includes model, retrieval, external API, compute, storage, and human review per successful task. Safety includes authorization violations, injection success, data leakage, harmful actions, and near misses. User satisfaction combines explicit ratings with task continuation, corrections, rework, escalation, and retention, interpreted carefully for selection bias.

Architecture and implementation

- Create exact metric definitions, denominators, and event sources in an evaluation contract.
- Measure both average and tail behavior and segment by risk and task complexity.
- Normalize cost and latency by successful or useful outcome, not request volume alone.
- Use hard safety gates and separate them from optimization metrics.
- Pair user feedback with objective completion and error signals.

Bottlenecks, risks, and safeguards

- Tool accuracy measured only when a tool is called ignores unnecessary calls.
- Minimal paths are not always safest; required validation adds legitimate steps.
- User ratings are sparse and biased toward extremes.
- Averages hide a small set of catastrophic or very expensive runs.

Validation and operating metrics

- A balanced scorecard with explicit thresholds and confidence intervals.
- Metric coverage: percentage of runs with complete, trustworthy instrumentation.
- Correlation between offline metrics, production outcomes, and human judgments.
- Regression and alert thresholds by task and risk tier.

What the interviewer is testing: Whether you can turn broad quality concepts into operational definitions that support decisions and alerts.

83. An agent reaches the correct result but uses twenty tool calls when five should be sufficient. How would you score this behavior?

The expected answer treats correctness as necessary but penalizes unnecessary risk and cost.

Model answer

I would mark the outcome correct but the trajectory inefficient. The penalty should consider excess cost, latency, rate-limit consumption, data exposure, and risk from extra side effects. I would compare the run with a reference policy, expert path, or lower-bound set of required capabilities. Because there may be

several valid paths, I would avoid exact path matching and instead score redundant or non-contributing actions.

A tool call contributes if it satisfies a dependency, adds material evidence, validates a claim, or executes a necessary action. Repeated searches with no information gain, duplicate lookups, or unnecessary model-tool cycles reduce the efficiency score. For high-risk tools, unnecessary use may also fail a safety gate even when the final answer is correct.

Architecture and implementation

- Classify each call as required, useful, redundant, failed but justified, or unsafe.
- Measure marginal state or evidence change after every call.
- Create task-specific safe-path baselines and cost envelopes rather than one universal optimum.
- Weight excess calls by cost, latency, data sensitivity, and side-effect risk.
- Use trace feedback to improve planning, tool descriptions, caching, and loop detection.

Bottlenecks, risks, and safeguards

- Penalizing all extra calls can discourage necessary verification and robustness.
- An expert path may not be unique or may assume unavailable knowledge.
- A cheap redundant call is different from an unnecessary payment or broad data query.
- Optimizing for minimum calls can produce brittle agents that do not recover from errors.

Validation and operating metrics

- Outcome score and separate efficiency score.
- Useful-call ratio, redundant cost, and marginal information gain.
- Safe-path regret in calls, time, and dollars.
- Task success after applying stricter efficiency policies.

What the interviewer is testing: Whether you can reward correct outcomes while recognizing that trajectory quality affects economics, reliability, and safety.

84. How would you create an evaluation dataset for an agent when there are many valid paths to complete each task?

This tests invariant-based evaluation, simulators, and path-agnostic rubrics.

Model answer

I would define each case with an initial environment state, user goal, constraints, available tools, hidden ground truth, allowed side effects, prohibited actions, and terminal success conditions. Instead of labeling one exact trajectory, I would specify invariants and acceptable end states. Optional reference paths can help diagnose efficiency, but the evaluator should allow alternative safe paths.

The dataset should include normal, ambiguous, adversarial, and failure-injected variants. A deterministic simulator or sandbox makes tool behavior reproducible and allows us to inspect state transitions. Human experts label semantic goals, escalation requirements, and acceptable uncertainty. Production failures and rare high-risk cases are continuously added with privacy controls.

Architecture and implementation

- Represent cases as executable scenarios with state, tools, events, and assertions.
- Define hard invariants, soft quality rubrics, and resource budgets separately.
- Include alternative valid outcomes such as completion, correct clarification, or justified abstention.

Create parameterized variations for permissions, tool failures, conflicting data, and timing.

Maintain train, development, hidden regression, and adversarial sets with leakage controls.

Bottlenecks, risks, and safeguards

A single gold path overfits the agent to evaluator conventions.

Static mock tools miss production timing and consistency failures.

LLM-generated cases can reproduce model biases and lack realistic edge conditions.

Using production traces without de-identification can expose customer data.

Validation and operating metrics

Scenario coverage by task, tool, risk, failure mode, and terminal state.

Inter-rater agreement and evaluator reproducibility.

Performance on hidden and newly added cases versus development cases.

Correlation of simulator results with production canaries.

What the interviewer is testing: Whether you can evaluate flexible behavior with end-state and invariant specifications rather than exact trace matching.

85. How would you test an agent's behavior when tools return errors, incomplete results, stale information, or contradictory data?

The expected answer uses fault injection and asserts recovery behavior, not only final text.

Model answer

I would build contract-level test doubles and scenario simulators that can inject typed failures at precise points: timeouts before and after side effects, rate limits, malformed schemas, partial pages, stale versions, permission denials, and conflicting sources. Each case defines expected state transitions, retry limits, escalation, user messaging, and side-effect invariants. The agent should not simply be judged on whether it eventually says something plausible.

Tests should include repeated and correlated failures, not only one isolated error. For stale or contradictory information, I would verify source-version handling, authority rules, dependency invalidation, and abstention. Chaos tests in a staging environment can introduce latency, worker loss, queue duplication, and provider outages to validate durability and idempotency.

Architecture and implementation

Create deterministic failure scripts for every tool adapter and workflow step.

Assert retries, backoff, circuit breakers, state, receipts, and user-visible status.

Inject ambiguous outcomes around side-effect boundaries.

Test stale, conflicting, and eventually consistent data with versioned fixtures.

Run periodic chaos and game-day exercises for cross-service failures.

Bottlenecks, risks, and safeguards

Happy-path mocks create false confidence.

Random chaos without assertions produces noise. Tie faults to expected control behavior.

An LLM may compensate in text while external state remains wrong.

Test doubles can drift from real APIs. Run contract and synthetic production tests.

Validation and operating metrics

Failure-mode coverage and recovery success by injected fault.

- Unproductive retries, duplicate actions, and incorrect terminal states.
- Circuit-breaker and degradation behavior under dependency failure.
- Differences between simulated and real incident traces.

What the interviewer is testing: Whether you test agent control behavior under realistic distributed-system failures and verify state, not just language.

86. What should be included in an agent execution trace?

This tests observability design, reproducibility, and privacy-aware logging.

Model answer

A trace should begin with task ID, parent or session ID, tenant, authenticated actor and delegated identity, workflow and policy versions, risk class, budgets, and timestamps. It should record each state transition, plan and revision, component version, model request metadata, selected context references, structured model output, tool candidates, authorization decisions, action arguments or protected hashes, tool attempts, results, receipts, validation findings, approvals, and terminal status.

Large or sensitive content should be stored in controlled artifact systems and referenced from the trace, with redaction and retention policy. The trace should preserve enough input and version information for replay without exposing secrets or private chain-of-thought. Correlation IDs must connect the orchestrator, model gateway, tool gateway, queues, and downstream services.

Architecture and implementation

- Use structured spans and events with stable schemas and causality links.
- Record prompt, model, tool, policy, and workflow versions plus configuration hashes.
- Capture state diffs and evidence references rather than only free-form logs.
- Apply field-level redaction, encryption, access control, sampling, and retention.
- Support search by task, action, error class, model, tool, tenant, and policy decision.

Bottlenecks, risks, and safeguards

- Full prompts and outputs can contain secrets or personal data.
- Sampling can omit the one trace needed for a serious incident. Retain high-risk and failed runs deterministically.
- Unstructured logs are difficult to replay or aggregate.
- Trace instrumentation can add latency and cost. Buffer asynchronously while preserving critical action events durably.

Validation and operating metrics

- Trace completeness, correlation success, and schema-valid event rate.
- Observability latency, storage cost, and sampling coverage.
- Redaction failures and unauthorized trace access.
- Mean time to diagnose using available traces.

What the interviewer is testing: Whether you can make agent decisions and side effects reconstructable while protecting sensitive content.

87. How would you trace a final answer back to the model calls, retrieved evidence, tool results, state changes, and approvals that produced it?

The expected answer builds provenance and claim lineage, not merely chronological logs.

Model answer

I would assign stable IDs to every artifact, claim, action, and state transition. Model outputs reference the input artifact versions they consumed. Retrieved evidence records source spans and versions. Tool results carry operation and provider IDs. The response generator produces structured claims with supporting evidence and action references before rendering prose. This creates a provenance graph from each final assertion back to sources and execution receipts.

Approvals link to exact action digests, and state transitions link to the events that caused them. A trace viewer can start from a sentence or action in the final answer and show supporting claims, evidence, model and prompt versions, validation findings, and unresolved uncertainty. For privacy, the viewer enforces the same access controls as the underlying sources.

Architecture and implementation

- Use a claim-evidence schema and artifact version IDs throughout the workflow.
- Record consumed and produced artifact references on every component span.
- Generate final output from approved structured claims and action-status records.
- Store dependency edges in a provenance graph or queryable lineage tables.
- Provide drill-down and replay tools with source-level authorization.

Bottlenecks, risks, and safeguards

- Chronological logs do not prove which evidence supported which claim.
- A generated citation may point to a relevant source that does not support the assertion.
- Derived summaries can break lineage if source spans are discarded.
- Lineage can expose restricted sources to viewers without equivalent permissions.

Validation and operating metrics

- Percentage of factual claims and action statements with valid lineage.
- Citation entailment and unsupported-claim rate.
- Lineage query latency and source-resolution success.
- Audit and incident cases resolved using provenance.

What the interviewer is testing: Whether you can explain causality and evidence at claim level rather than offering a large undifferentiated log.

88. An LLM judge prefers verbose agent responses even when they contain unsupported claims. How would you reduce judge bias?

This tests evaluator calibration and decomposition of quality dimensions.

Model answer

I would split evaluation into dimensions and use objective checks before the judge. Claim extraction and citation verification measure support. Tool receipts verify actions. Deterministic tests verify calculations and schemas. The LLM judge receives a concise rubric for relevance, completeness, and communication, with explicit instruction not to reward length. It compares answers with evidence visible and returns structured findings, not only a scalar preference.

I would calibrate the judge against a human-labeled set containing concise correct answers, verbose unsupported answers, and adversarial style variations. Position and length are randomized or normalized. Multiple judges or models can be used where their errors are sufficiently independent, and disagreement routes to human review. Judge versions are tracked like production models.

Architecture and implementation

- Evaluate factual support, action truth, safety, and style as separate dimensions.
- Use pairwise and pointwise calibration examples that expose verbosity bias.
- Require evidence-linked reasons and allow an insufficient-evidence outcome.
- Randomize answer order and control length in benchmark experiments.
- Monitor judge-human agreement, subgroup behavior, and drift after judge updates.

Bottlenecks, risks, and safeguards

- A judge using the same model family can share producer errors.
- Long rubrics can be inconsistently followed. Use a small number of well-defined criteria.
- Majority voting among biased judges does not remove bias.
- Optimizing against a known judge can lead to reward hacking.

Validation and operating metrics

- Judge-human agreement, precision for unsupported claims, and length correlation.
- Inter-judge disagreement and calibration by answer style.
- Performance on hidden adversarial evaluator cases.
- Regression in production outcomes after optimizing to judge scores.

What the interviewer is testing: Whether you can treat an LLM judge as a fallible measurement instrument and calibrate it against objective checks and humans.

89. How would you create adversarial tests for prompt injection, authorization bypass, infinite loops, duplicate actions, and memory poisoning?

The expected response is a systematic threat-informed suite with measurable attack success.

Model answer

I would build an attack matrix across entry points, trust boundaries, tools, and consequences. Prompt-injection cases appear in web pages, documents, tool fields, images, code comments, and peer-agent messages, including obfuscation and multilingual variants. Authorization cases test forged tenant IDs, role claims, indirect tool chains, stale approvals, and direct gateway access. Loop cases vary queries and state to evade exact repetition. Duplicate-action cases inject timeouts at every transaction boundary. Memory cases insert false, conflicting, cross-tenant, and high-sensitivity claims.

Each scenario has a required safe outcome and instrumentation assertions: the action must be blocked, trust labels preserved, operation ID reused, memory quarantined, or run terminated correctly. Tests run at component, workflow, sandbox, and red-team levels. Production incidents become permanent regressions, and mutation or fuzzing generates variants around known weaknesses.

Architecture and implementation

- Map threats to assets, trust boundaries, attacker capabilities, and expected controls.
- Create executable scenarios with deterministic state and side-effect assertions.
- Use automated fuzzing plus human red teaming for novel semantic attacks.
- Run tests against every model, prompt, tool, and policy bundle before release.

Track vulnerabilities, remediation, and regression ownership in the deployment process.

Bottlenecks, risks, and safeguards

A fixed benchmark becomes easy to overfit. Keep hidden cases and generate mutations.

Testing only final responses misses attempted unsafe tool calls.

Synthetic attacks may not reflect real user workflows. Incorporate sanitized incidents and domain experts.

Red-team content can contain sensitive exploit details; control access and retention.

Validation and operating metrics

Attack success rate by threat, model, tool, and defense layer.

Detection and containment latency plus false-positive rate.

Regression coverage and recurrence of previously fixed classes.

Depth of impact prevented, from model output through actual side effect.

What the interviewer is testing: Whether you can turn agent threat models into executable, layered, continuously evolving security evaluations.

90. A new model improves average agent performance but causes regressions in several critical workflows. How would you design deployment gates?

This tests risk-weighted evaluation, canarying, shadowing, and selective routing.

Model answer

I would not deploy based on average score. Each critical workflow has hard regression gates for task success, authorization, side-effect correctness, structured output, latency, and cost. The new model must pass a hidden scenario suite, adversarial tests, and replay against representative production traces. Results are segmented by workflow, language, tenant requirements, and tool pattern.

Deployment proceeds through shadow mode, offline comparison, low-risk canary, and gradually expanded routing. The model may be enabled only for workflows where it improves or remains within tolerance, while critical regressions stay on the previous route. Automatic rollback uses error, safety, latency, and business thresholds. Prompts and tool descriptions are evaluated as a bundle because model changes can alter their interpretation.

Architecture and implementation

Define non-negotiable gates and allowed trade-off bands per workflow and risk tier.

Replay stored contexts with tool outputs and run live shadow decisions without side effects.

Canary by task class, tenant, and percentage with capacity-aware fallback.

Use feature flags and versioned routing for immediate rollback.

Investigate regressions and adapt prompts or validators only if changes preserve other workflows.

Bottlenecks, risks, and safeguards

Shadow runs cannot reveal all effects of changed actions and downstream state.

A global rollout hides task-specific failures behind average improvement.

Fallback can overload the old model during rollback. Reserve capacity.

Changing model and prompt simultaneously weakens attribution. Use controlled bundles and ablations.

Validation and operating metrics

Gate pass rate and regression severity by critical workflow.

Canary delta in success, safety, latency, cost, and user corrections.

Rollback trigger accuracy and time to restore stable routing.

Coverage of production traffic by evaluated scenarios.

What the interviewer is testing: Whether you can protect critical workflows with segmented gates and staged rollout instead of optimizing a global average.

SECTION 10

Performance, Scaling, Cost, and Bottlenecks

These questions cover latency decomposition, rate limits, queueing, million-task scale, durable state, noisy-neighbor controls, model routing, caching, and graceful degradation.

Section focus: Optimize the full critical path and the cost per successful task. Agent systems are commonly bottlenecked by queues, tools, serialization, retries, and context construction rather than model inference alone.

91. An agent takes 30 seconds to complete a task. How would you decompose latency across planning, model inference, retrieval, tool calls, validation, and response generation?

The interviewer expects a critical-path trace and a measurement-first optimization plan.

Model answer

I would instrument the request as a distributed trace with spans for admission and queueing, context construction, each model call, retrieval, reranking, tool gateway, downstream service, validation, persistence, and response rendering. For parallel operations, I would distinguish total work from critical-path time. I would report p50, p95, and p99 because an average can hide slow tools or retries.

Then I would inspect sequential dependencies, idle gaps, repeated calls, and retry amplification. A 30-second run may contain ten seconds of tool wait, eight seconds of model calls, five seconds of queueing, and seven seconds of unnecessary serialization. Optimization follows evidence: parallelize independent reads, remove redundant planning, stream early status, cache stable retrieval, use a smaller model for routing, set deadlines, and replace slow tools or add asynchronous execution where appropriate.

Architecture and implementation

- Propagate one trace ID through API, workflow, model gateway, retrieval, tool gateway, and downstream services.
- Record queue delay, service time, retries, payload size, token counts, and dependency edges for every span.
- Build critical-path and waterfall views by task class and model route.
- Set component-level deadlines from the end-to-end latency budget.
- Optimize the largest measured contributor and rerun quality and safety tests after each change.

Bottlenecks, risks, and safeguards

- Adding span durations without dependency information double-counts parallel work.
- Aggressive parallelism can increase rate limits, cost, and data exposure.
- Reducing validation may improve latency but create false completion or unsafe actions.
- Caching can introduce stale or cross-tenant results if keys are incomplete.

Validation and operating metrics

- End-to-end and component p50, p95, and p99 latency.
- Critical-path percentage by component and idle or queue time.
- Retries, model calls, and tool calls per task.
- Quality, safety, and cost change after each latency optimization.

What the interviewer is testing: Whether you can diagnose latency from traces and optimize the critical path without guessing or removing necessary controls.

92. Model calls account for only 20% of total latency. What other bottlenecks would you investigate?

This tests awareness of non-model bottlenecks in a distributed agent platform.

Model answer

I would examine queue admission, cold starts, workflow persistence, context retrieval, vector and lexical search, reranking, tool discovery, tool gateway policy checks, downstream APIs, database locks, network hops, serialization, large payloads, retries, and human approval. I would also look for artificial serialization: a plan may call independent tools one after another or wait for a slow optional branch before producing a useful response.

At scale, connection pools, thread or worker exhaustion, rate-limit backoff, cache misses, and tenant contention often dominate. For long tasks, checkpoint and artifact-upload time can matter. The first step remains tracing by task type and percentile. A bottleneck that affects only p99 high-risk workflows requires a different fix from one that affects every request.

Architecture and implementation

- Measure queue, compute, storage, network, policy, and external-dependency spans separately.
- Inspect saturation signals: utilization, queue depth, connection pools, locks, and throttles.
- Identify optional versus mandatory critical-path operations.
- Profile payload and token construction, compression, and repeated serialization.
- Test parallelism, caching, connection reuse, and asynchronous handoff under realistic load.

Bottlenecks, risks, and safeguards

- Optimizing the model while a slow API dominates yields little user benefit.
- High averages can come from a small subset of tool calls; segment before redesigning the platform.
- Increasing concurrency can overload downstream systems and worsen tail latency.
- Moving work off the synchronous path can reduce perceived latency but not completion time; report both.

Validation and operating metrics

- Queueing, saturation, cache, network, and tool latency by percentile.
- Cold-start frequency and worker or connection utilization.
- Sequential versus parallel wait and optional-work contribution.
- User-perceived first response, useful-result time, and final completion time.

What the interviewer is testing: Whether you think like a distributed-systems engineer and investigate the full request lifecycle.

93. A tool used by the agent has a strict rate limit. How would you implement queuing, throttling, batching, caching, and backpressure?

The answer should protect the dependency and preserve fairness and priority.

Model answer

I would centralize calls through the tool gateway so the platform has one view of the provider quota. A token-bucket or leaky-bucket limiter applies global, tenant, and operation-specific rates. Requests enter a bounded priority queue with deadlines and fair scheduling. Interactive, high-value, or recovery operations may receive higher priority, but no tenant can consume the full quota.

Read requests are deduplicated and cached according to data freshness and authorization. Batchable operations are grouped within a short window while preserving per-item identity and error handling. When demand exceeds sustainable capacity, the system applies backpressure at admission: defer long tasks, reduce optional work, use an approved alternate source, or reject with a retry-after status. Agents should not independently retry and multiply load.

Architecture and implementation

- Enforce rate limits and retry budgets at the shared gateway, not in each agent.
- Use bounded queues, deadlines, priorities, and weighted fair scheduling.
- Cache read-only results with tenant, permissions, query, version, and freshness in the key.
- Batch only semantically compatible calls and preserve idempotency and per-item receipts.
- Expose capacity signals to planners so they can choose defer, alternate, or partial paths.

Bottlenecks, risks, and safeguards

- Unbounded queues convert rate limits into extreme latency and memory exhaustion.
- Retries from many workers can synchronize into a thundering herd. Use centralized budgets and jitter.
- Caching can leak or return stale data. Include authorization and versioning and set domain-specific TTLs.
- Batch failure may obscure which items succeeded. Require item-level status and idempotency.

Validation and operating metrics

- Quota utilization, throttle rate, queue depth, wait time, and deadline expiry.
- Cache hit, deduplication, and batch efficiency.
- Fairness and starvation by tenant and priority class.
- Provider errors and agent task success under sustained overload.

What the interviewer is testing: Whether you can turn a scarce external dependency into a governed shared resource with fairness and graceful overload behavior.

94. How would you scale an agent platform supporting one million concurrent long-running tasks?

This tests durable orchestration, partitioning, event-driven workers, and control-plane versus data-plane separation.

Model answer

I would not keep one million agents as in-memory processes. Each task is durable state plus scheduled work. A horizontally partitioned workflow service stores task metadata, event history, timers, and next-ready steps. Stateless workers pull executable activities from queues, obtain short leases, perform model or tool

work, write results, and release capacity. Most tasks are dormant while waiting for events and consume storage, not compute.

The platform separates control plane services - identity, policy, registry, routing, budgets, and orchestration - from data plane workers for models, tools, retrieval, and sandboxes. Tasks are sharded by tenant or task ID, queues by capability and priority, and artifacts stored in scalable object storage. Backpressure, admission quotas, regional placement, and per-tenant isolation prevent overload. Multi-region design requires ownership rules so one task has one active coordinator.

Architecture and implementation

- Use durable workflow or event-sourced state with timers and signals, not sleeping threads.
- Partition task state and queues, and use stateless autoscaled workers with leases and fencing.
- Separate ready, waiting, approval, callback, and terminal states to avoid unnecessary polling.
- Tier storage: hot task state, append-only events, object artifacts, and archival traces.
- Apply admission control, quotas, priority, regional affinity, and bulkhead isolation.

Bottlenecks, risks, and safeguards

- Central schedulers or hot partitions can bottleneck at timer or state-update volume.
- At-least-once queue delivery requires idempotent activities and action reconciliation.
- Mass wake-ups from a shared timer or provider recovery can create a thundering herd.
- Workflow version changes across millions of active runs require compatible replay and migration.

Validation and operating metrics

- Active, ready, waiting, and terminal task counts plus transition throughput.
- Queue delay, worker utilization, lease expiry, and duplicate activity rate.
- State-store latency, hot-partition skew, and event growth.
- Cost per active and completed task, recovery time, and regional failover success.

What the interviewer is testing: Whether you model long-running agents as durable workflows and can scale state transitions independently from compute.

95. Would you store long-running agent state in memory, a relational database, an event store, a workflow engine, or a combination?

The expected answer assigns each option a role and discusses consistency, replay, and operational complexity.

Model answer

I would use a combination. Process memory may cache active state but cannot be authoritative. A workflow engine or orchestration service manages timers, retries, signals, and durable execution. A relational or strongly consistent state store holds the current materialized task, ownership, budgets, approvals, and queryable status. An append-only event store preserves transitions for audit, replay, and rebuilding state. Object storage holds large artifacts and tool outputs.

The exact product choice depends on throughput, task duration, query requirements, and operational maturity. A workflow engine can encapsulate the event history and timers, but I may still maintain an application-level task table for product queries. State ownership and transaction boundaries must be clear to avoid two competing sources of truth.

Architecture and implementation

- Keep ephemeral caches disposable and rebuildable.

- Use durable orchestration for waits, retries, callbacks, and versioned execution.
- Maintain a query-optimized current-state view with optimistic concurrency or transactions.
- Persist immutable events or workflow history for reconstruction and audit.
- Store large artifacts by reference with checksum, version, and access policy.

Bottlenecks, risks, and safeguards

- Memory-only state is lost on crash and cannot support safe worker migration.
- Dual writes to event and state stores can diverge. Use transactional outbox, workflow events, or one authoritative commit path.
- Event replay can be slow without snapshots.
- A general workflow engine may impose history or payload limits; externalize large data.

Validation and operating metrics

- State transition latency, consistency errors, and snapshot or replay time.
- Workflow history size and archival cost.
- Recovery correctness after process and regional failures.
- Operational burden and incident rate by storage component.

What the interviewer is testing: Whether you can compose durable execution, current state, event history, and artifact storage without ambiguous ownership.

96. How would you prevent noisy tenants from consuming all available model and tool capacity?

This tests multi-tenant fairness, quotas, bulkheads, and priority admission.

Model answer

I would enforce quotas at admission and at each scarce dependency. Each tenant has concurrency, request-rate, token, tool-call, sandbox, and spending limits, potentially by user tier and task class. Weighted fair queues and reserved capacity prevent one tenant from occupying all workers. High-cost fan-out is estimated before admission and charged against a budget rather than discovered after execution.

Bulkheads isolate critical workflows and high-risk recovery traffic from best-effort jobs. Rate limits are applied at the model gateway, tool gateway, retrieval, and code-execution layers using trusted tenant identity. When limits are reached, the platform defers, degrades, or rejects with transparent status. Enterprise reservations can be supported without permitting unlimited bursts that destabilize shared dependencies.

Architecture and implementation

- Propagate authenticated tenant identity into every queue, cache, worker, and provider call.
- Use hierarchical quotas: organization, project, user, task, model, and tool.
- Schedule with weighted fairness, concurrency caps, and reserved pools for critical workloads.
- Estimate and reserve fan-out or token cost before launching parallel work.
- Expose usage dashboards, alerts, and administrative controls to tenants.

Bottlenecks, risks, and safeguards

- Per-request rate limits miss one task that creates hundreds of subcalls.
- Shared caches or queues without tenant labels can bypass accounting or leak data.
- Strict caps can block incident-response tasks. Support policy-controlled priority and break-glass capacity.
- Tenant sharding can create stranded capacity; combine logical fairness with elastic worker pools.

Validation and operating metrics

- Capacity and cost by tenant, task, model, and tool.
- Fairness, queue wait, throttle rate, and starvation.
- Budget prediction error for fan-out tasks.
- Impact of a tenant load spike on other tenants p95 latency and success.

What the interviewer is testing: Whether you can provide multi-tenant fairness across recursive agent workloads and all downstream bottlenecks.

97. A stronger model improves task completion by 5% but triples cost. How would you determine when to use the stronger model?

The expected answer uses expected value, task segmentation, and escalation rather than one global model choice.

Model answer

I would measure the improvement by task class, risk, and complexity because the average 5% may be concentrated in difficult cases. Then I would compare incremental cost with the business value of a successful task, cost of failure, human fallback cost, and latency impact. High-risk or high-value decisions may justify the stronger model even for a small lift; routine transformations may not.

I would build a routing or escalation policy. Simple, low-risk cases start with a smaller model. A stronger model is selected when input complexity, uncertainty, failed validation, tool-selection ambiguity, or historical error likelihood exceeds a threshold. For some workflows, the strong model is used only for planning or final review. Online canaries and cost-quality frontiers validate the policy.

Architecture and implementation

- Segment evaluation results and estimate expected utility per model and task type.
- Use features such as input length, novelty, risk, confidence, and prior step failure for routing.
- Support escalation after a cheap attempt only when the first attempt has not caused irreversible action.
- Reserve strong-model capacity for tasks with highest marginal value.
- Continuously recalibrate routing as prices, latency, and model quality change.

Bottlenecks, risks, and safeguards

- A cheap-first strategy can double latency and cost if escalation is common.
- Model confidence may not predict failure well. Use calibrated task features and validator signals.
- Average quality lift can hide regressions in critical workflows.
- Routing can become self-reinforcing if hard cases sent to one model are judged differently.

Validation and operating metrics

- Success, cost, and latency by route and task segment.
- Routing regret compared with the best feasible model for each case.
- Escalation rate and incremental success from escalation.
- Business value or failure cost avoided per additional model dollar.

What the interviewer is testing: Whether you can optimize expected value with selective model use instead of choosing the strongest or cheapest model everywhere.

98. How would you implement model routing so that simple steps use smaller models while complex or high-risk steps use stronger models?

This tests route classification, capability contracts, fallback, and evaluation.

Model answer

Each agent node should request a capability and quality tier rather than name a model. A routing service combines task type, input complexity, context length, tool requirements, risk class, latency target, data policy, model health, and historical performance. Hard constraints filter eligible deployments; a policy or learned scorer selects among them. High-risk nodes may have an allowlist and mandatory validator regardless of model strength.

Routing can be static by node initially, then become adaptive. For example, intent classification and summarization use small models, while ambiguous planning, conflict resolution, or high-impact review use stronger models. A failed schema, low-confidence result, or validator finding may trigger escalation. Every route and fallback is versioned and recorded so quality can be attributed.

Architecture and implementation

- Define capability contracts for structured output, tool use, modality, context, region, and quality tier.
- Use deterministic eligibility filters followed by a cost-quality-latency scorer.
- Calibrate escalation triggers from validation and task features.
- Canary routing-policy changes and preserve route-specific fallback capacity.
- Prevent a fallback from violating data residency, safety, or tool-compatibility requirements.

Bottlenecks, risks, and safeguards

- A small model may produce a plausible but wrong routing classification. Use conservative thresholds for high-risk work.
- Escalation after side effects may be too late. Route critical planning and approval correctly before execution.
- Provider-specific tool semantics can break model substitution.
- Dynamic routing complicates reproducibility. Record all features, policy version, and selected deployment.

Validation and operating metrics

- Route accuracy, task success, latency, and cost by node and task class.
- Escalation precision, recall, and success lift.
- Fallback rate, capacity saturation, and routing-policy stability.
- Safety and compliance results by selected model route.

What the interviewer is testing: Whether you can create a governed capability router with safe escalation and full attribution.

99. Which agent operations can be cached safely, and how would you construct cache keys for plans, tool selections, retrieval results, and generated outputs?

The expected answer distinguishes deterministic or read-only work from personalized, stateful, and side-effecting operations.

Model answer

Safe cache candidates include immutable document parsing, embeddings, stable retrieval candidates, deterministic calculations, tool metadata, and some model outputs for identical public inputs. Plans or tool selections can be cached only when the task specification, capability catalog, policy, permissions, model

and prompt versions, and relevant state are equivalent. Side-effecting tool calls are not result-cached as a substitute for idempotency, though their operation receipts are stored for deduplication.

Cache keys should be hashes of canonical inputs plus all variables that can change correctness or access: tenant and user scope, purpose, authorization, source versions, query, time or freshness window, locale, model deployment, prompt, parameters, tool registry, workflow, and policy. Entries carry provenance, TTL, invalidation dependencies, and sensitivity. The system should prefer recomputation over a cache hit when equivalence cannot be proven.

Architecture and implementation

- Separate content-addressed immutable caches from time-based or permission-sensitive caches.
- Include identity and authorization scope in any cache containing private data.
- Track dependencies so document, policy, prompt, model, or tool changes invalidate entries.
- Use semantic caching cautiously with similarity thresholds and answerability or scope checks.
- Store action receipts by operation ID for idempotent replay rather than caching arbitrary write responses.

Bottlenecks, risks, and safeguards

- A missing tenant or policy dimension can cause data leakage or unauthorized reuse.
- Cached plans can become invalid when tools or external state change.
- Semantic cache matches may be similar in language but different in constraints.
- Caching model output can preserve an old safety or factual error after the system is fixed.

Validation and operating metrics

- Hit rate, latency and cost savings, and stale-result rate by cache type.
- Cross-tenant isolation and authorization-key coverage.
- Invalidation delay and cache-related incidents.
- Quality difference between cached and freshly computed outcomes.

What the interviewer is testing: Whether you can reason about cache equivalence in a versioned, personalized, policy-controlled agent system.

100. Design graceful-degradation behavior when the primary model, memory store, workflow engine, tool gateway, or external API is unavailable.

This tests dependency-specific degradation, safe failure, and avoidance of silent capability loss.

Model answer

Each dependency needs an explicit degradation contract. If the primary model fails, the gateway can route to a validated fallback that satisfies the task's tool, data, region, and quality requirements; otherwise the task pauses or returns a limited response. If memory is unavailable, the agent can continue only for tasks that do not depend on durable personalization, while clearly avoiding memory-based claims. If the workflow engine is unavailable, new long tasks should not start; stateless read-only requests may use a separate path, and in-flight tasks remain durable for recovery.

If the tool gateway is unavailable, no external action should be claimed or attempted through a bypass. The system may provide informational guidance from approved sources. If one external API is down, use an approved alternate, cached read within freshness limits, manual queue, or partial completion. Degradation should preserve safety and truthfulness even if functionality drops.

Architecture and implementation

- Define dependency health, circuit breakers, fallback eligibility, and user-visible status per workflow.

Separate read-only informational paths from transactional action paths.
Reserve fallback capacity and test it regularly through game days.
Queue or checkpoint deferred work with expiration, cancellation, and deduplication.
Never bypass authorization, validation, or audit controls to improve availability.

Bottlenecks, risks, and safeguards

Fallback models may not support the same tool schema, region, or safety behavior.
Cached external data can exceed acceptable staleness for high-impact decisions.
A hidden bypass around the tool gateway destroys audit and authorization guarantees.
Mass fallback can overload secondary systems and create cascading failure.

Validation and operating metrics

Availability and task success by degraded mode.
Failover time, fallback error rate, and secondary-capacity saturation.
Tasks paused, partially completed, or incorrectly continued during outages.
Safety, authorization, and truthfulness incidents during degradation, expected to remain at baseline or better.

What the interviewer is testing: Whether you can reduce functionality without reducing safety, authorization, or honesty, and whether failover is capacity-tested rather than theoretical.

Final Interview Checklist

- I clarified risk, autonomy, scale, latency, and compliance before proposing components.
- I identified which decisions must be deterministic and which can be model-driven.
- I defined state, tool contracts, permissions, idempotency, and approval boundaries.
- I covered termination, recovery, replay, graceful degradation, and rollback.
- I included evaluation for outcome quality, path efficiency, safety, latency, and cost.
- I discussed prompt injection, data exfiltration, tenant isolation, and auditability.
- I explained the main bottleneck and the first optimization I would test rather than prematurely optimizing everything.

Closing principle: A production agent is not simply an LLM with tools. It is a governed distributed system in which probabilistic decisions are bounded by deterministic controls, explicit trust boundaries, and measurable outcomes.